Cofinancé par
l'Union européenne

Bocar SALL

3e année BUT GEII (Aix-Marseille Université)

Departement GEII

Mise en fonctionnement du robot potager « farmbot » et implémentation d'une intelligence artificielle pour la surveillance à distance des maladies sur des plantes.

Stage de fin d'etude – BUT3 GEII | Departement GEII

Rapport de stage

Année 2024-2025

Stage effectué du 24/MARS/2025 au 13/JUIN/2025

IUT Aix Marseille Université, 142 trav. Charles Susini,13013 Marseille, France

Tuteur de stage : **AUBEPART Fabrice**

Tutrice enseignante : **CELLIER Marine**

Résumé

Durant ce stage, j'ai travaillé sur l'automatisation et l'amélioration de la surveillance des cultures à l'aide du robot potager **Farmbot**. La première étape de mon travail a consisté en la mise en service du robot, incluant l'analyse de sa documentation technique ainsi que la configuration logicielle à partir des outils fournis par le fabricant. Cela m'a permis de comprendre le fonctionnement global du système et d'assurer son bon démarrage.

Une fois le robot opérationnel, j'ai intégré une solution d'intelligence artificielle visant à détecter certaines maladies végétales à partir d'images capturées automatiquement par la caméra de ma Raspberry Pi. Pour cela, j'ai utilisé un modèle que j'ai entraîné sur **Teachable Machine**. Ce modèle a ensuite été intégré sur le Raspberry Pi, permettant une détection automatique des anomalies sur les feuilles des plantes.

Enfin, pour permettre une supervision à distance, j'ai mis en place un système de transmission des données via le protocole **LoRaWAN**. Cette technologie de communication longue portée et basse consommation est particulièrement adaptée aux environnements agricoles. Grâce à cette intégration, les données (état sain ou malade des plantes) peuvent être consultées à distance par l'utilisateur, facilitant ainsi une réaction rapide en cas de problème.

Remerciements

Je tiens tout d'abord à remercier toute l'équipe pédagogique du département GEII de l'IUT d'Aix-Marseille pour la qualité des enseignements qui m'ont été transmis tout au long de cette annee.

Je tiens à exprimer ma gratitude à l'ensemble de l'équipe de la Direction des Relations Internationales, et plus particulièrement aux personnes en charge des étudiants en programme d'échange ERASMUS, pour leur grande disponibilité.

Je tiens à exprimer ma plus sincère reconnaissance à Monsieur Fabrice AUBEPART, mon tuteur pédagogique et maître de stage, pour avoir été disponible tout au de cette expérience. En tant qu'étudiant en échange entre l'ESP de Dakar et l'IUT d'Aix-Marseille, je lui suis particulièrement reconnaissant de m'avoir aidé à trouver ce stage et de m'avoir accompagné avec rigueur, et bienveillance durant toute sa réalisation.

J'exprime aussi ma gratitude envers l'École Supérieure Polytechnique de Dakar, pour les deux années de formation enrichissantes que j'y ai passées. Ces années ont constitué une base solide qui m'a permis d'aborder cette mobilité et ce stage.

Table des matières

Résumé 2
Remerciements 3
Table des matières 4
Introduction General 7
Chapitre I : Presentation du système 8
INTRODUCTION 8
1-Hardware 10
2-Software 12
3- Dépannage 21
Chapitre II : Mise en place du système de surveillance des plantes avec Raspberry Pi 23
INTRODUCTION 23
1. Matériel utilisé 23
2. Mise en place de la caméra 24
3. Capture des photos 25
3.1 Algorithme 25
3.2 scripte de la capture 26
Chapitre III : Prédiction d'images à l'aide d'un modèle IA 27
INTRODUCTION 27
1.Téléchargement du modèle IA 27
2.Installation des bibliothèques nécessaires au traitement 29
2.1 numpy (version 1.26.4) 29
2.2 pillow et tflite-runtime 29
3.Implémentation du modèle IA 29
L'algorigramme 29
4. Tableau de bord en ligne (Flask) 31
5.L'algorithme de l'ensemble du Système 33
Chapitre IV : Utiliser Lora Shield et Raspberry Pi pour construire une passerelle LoRaWAN 34
INTRODUCTION 34
1. Le protocole LoRaWAN 34

1.1 Différence entre LoRa et LoraWAN 34
1.2 Structure du reseau LoRaWAN 34
1.2.1 Les Devices LoRaWAN 35
1.2.2 Les Gateways LoRaWAN 35
1.2.3 Le Network Server 35
1.2.4 L'Application Server 36
1.3 Les différentes classes 36
1.3.1 Classe A (tous) : Application basse consommation 36
1.3.2 Classe B (Beacon) : Créneau de réception programmé 36
1.3.3 Classe C (Continu) : Écoute en permanence 37
1.4 Activation des devices LoRa ABP ou OTAA 37
1.4.1 ABP : Activation By Personalization 38
1.4.2 OTAA : Over The Air Activation 38
2. Version de notre reseau LoRaWAN 39
2.1 Le matériel Utilise 39
2.2 Mise en place 40
2.3 Cree un compte sur le reseau TTN 41
2.4 Créer une passerelle sur le reseau TTN 41
2.5 Ajouter le End device sur le TTN 42
2.5.1 Communication du Dragino et le Raspberry 42
2.5.2 Récupération du JoinEUI du module dragino 42
2.5.2.1 Création d'un environnement de travail 42
2.5.2.2 Téléchargement d'un forwarder LoRa monocanal 43
2.5.2.3 Compilation du forwarder et connexion du matériel 44
2.6 Essais 49
2.6.1 Détection IA des plantes sur Raspberry Pi (Chapitre III) 49
2.6.2 Transmission LoRa avec module Dragino 49
2.6.3 Fusion des deux scripts 49
3. Migration de TTN vers ChirpStack et Intégration Node-RED 50
3.1 Presentation de ChirpStack et Node-RED 50
3.2 Fonctionnement 51

Conclusion General..... 53
Reference..... 54
Annexes 55

Introduction General

L'évolution rapide des technologies numériques ouvre de nouvelles perspectives dans de nombreux domaines, notamment dans l'agriculture. La nécessité d'optimiser la production tout en réduisant l'impact environnemental conduit à développer des solutions innovantes. Dans ce contexte, mon stage s'inscrit dans un projet visant à automatiser et améliorer la surveillance des cultures grâce au robot potager FarmBot.

Présentation du lieu de stage

Mon stage a été réalisé au sein du département Génie Électrique et Informatique Industrielle (GEII) de l'IUT d'Aix-Marseille Universite, situé sur le campus de Marseille Saint-Jérôme. Le département GEII forme des techniciens dans les domaines de l'électronique et système embarques. L'encadrement pédagogique y est dynamique et encourage les projets intéressants, permettant aux étudiants de développer des compétences techniques solides et adaptées aux besoins industriels actuels.

Projet Africonnect+

Ce stage s'inscrit dans le cadre du projet **Africonnect+**, un programme de coopération européenne visant à renforcer les échanges académiques, scientifiques et technologiques entre l'Europe et plusieurs pays africains. Africonnect+ a pour but de soutenir le développement de solutions innovantes.

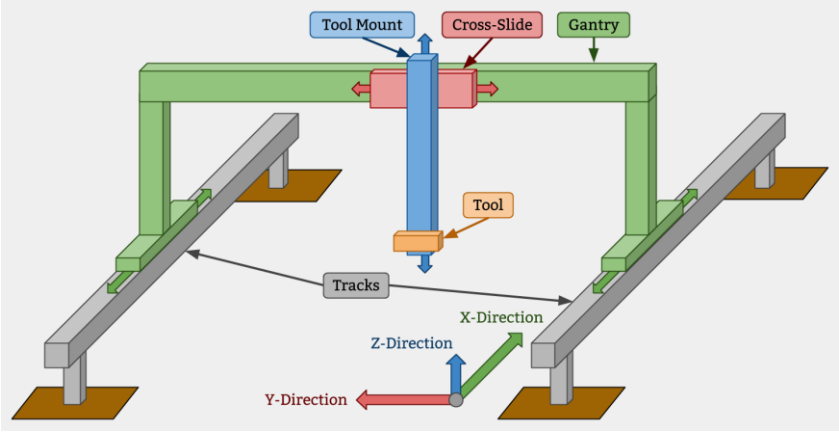
Le projet **AfriConnect+** s'inscrit dans la thématique européenne sur la transformation du numérique des pays dans les régions de la Méditerranée, notamment au Maroc, et de l'Afrique subsaharienne, au Sénégal. **AfriConnect+** s'inscrit dans la stratégie et l'objectif de modernisation et de développement des établissements d'enseignement supérieur au Sud qui partagent les objectifs communs concernant le développement de l'entrepreneuriat, les compétences professionnelles, l'innovation et le transfert technologique dans le domaine du numérique et plus particulièrement dans celui de l'Internet des objets (IoT).

Originaire du Sénégal j'ai eu l'opportunité de participer à ce programme dans le cadre d'un échange universitaire entre mon établissement d'origine au Sénégal (ESP de Dakar) et l'IUT d'Aix-Marseille. Cette expérience m'a permis d'approfondir des compétences techniques tout en découvrant un nouvel environnement académique et culturel, en lien direct avec les objectifs du projet Africonnect+.

Chapitre I : Presentation du système

INTRODUCTION

Farmbot est un projet open source d'agriculture de précision à commande numérique par ordinateur. Il comprend un robot agricole à coordonnées cartésiennes, un logiciel et une documentation, incluant un référentiel de données agricoles. L'objectif du projet est de « créer une technologie ouverte et accessible permettant de cultiver de la nourriture pour tous ». FarmBot est un projet open source permettant aux utilisateurs de modifier et d'ajouter du matériel, des logiciels et de la documentation.



La genèse du FarmBot

- Le FarmBot Genesis est capable de planter plus de 30 cultures différentes, notamment des pommes de terre, des pois, des courges, des artichauts…, sur une superficie de 2,9 mètres × 1,4 mètre avec une hauteur de plante maximale de 0,5 mètre.
- Il peut cultiver une variété de cultures dans la même zone en même temps et est capable de fonctionner à l'intérieur (figure1), à l'extérieur(figure2) ou dans des zones couvertes(serres)comme le montre la figure 3.

Figure1 : Prototype FarmBot Genesis V0.6 fonctionnant à l'intérieur.

Figure2 : Prototype FarmBot Genesis V0.7 fonctionnant à l'extérieur.

Figure3 : Un FarmBot placé à l'intérieur d'une serre permettant une culture toute l'année.

- Le Farmbot Genesis peut effectuer presque tous les processus avant la récolte, y compris le semis, le désherbage mécanique et l'arrosage.
- Cela nécessite de l'électricité, une connexion Internet et un approvisionnement en eau qui doivent être fournis à l'aide de solutions hors réseau, notamment un baril d'eau pour recueillir la pluie et un panneau solaire et une batterie pour fournir de l'électricité.
- Le FarmBot Genesis est capable de collecter des données pour prendre en compte des facteurs tels que les conditions météorologiques locales à partir de capteurs locaux et de données externes provenant d'Internet.

Figure4 : Le prototype FarmBot arrosant pour la première fois.

1-Hardware

Moteurs

FarmBot Genesis utilise des moteurs pas à pas NEMA 17 avec encodeurs intégrés pour un contrôle en boucle fermée. Ces encodeurs corrigent en temps réel les décalages, en évitant les pas manqués et assurant une précision optimale pour l'arrosage ou le désherbage. Contrairement à une boucle ouverte, ce système s'adapte aux variations de charge et détecte les blocages, améliorant la fiabilité. Les encodeurs, prémontés et testés, sont pilotés via le firmware open-source de Farmbot, alliant simplicité et robustesse pour un jardinage

Figure 5 : moteur Nema17

Outils FarmBot Genesis

- **Outil Rotatif Genesis**

Un accessoire polyvalent équipé d'un disque de coupe, idéal pour le désherbage de précision, l'aération du sol ou le traçage de sillons. Compatible avec la détection de résistance, il évite les blocages grâce au contrôle en boucle fermée des moteurs.

Figure6 : Outils rotatifs genesis

- **Buse d'Arrosage Genesis**

Conçue pour l'irrigation localisée, cette buse se connecte au système hydraulique de farmbot. Elle permet un arrosage programmable (débit et zone) et peut être couplée à des capteurs d'humidité pour un apport en eau optimisé.

Figure7 : Buse d'arrosage genesis

- **Capteur de Sol Genesis**

Ce capteur mesure l'humidité, la température et parfois la conductivité du sol. Intégré à l'interface logicielle, il permet une agriculture basée sur les données en déclenchant des actions (arrosage, alertes) basées sur des seuils personnalisés.

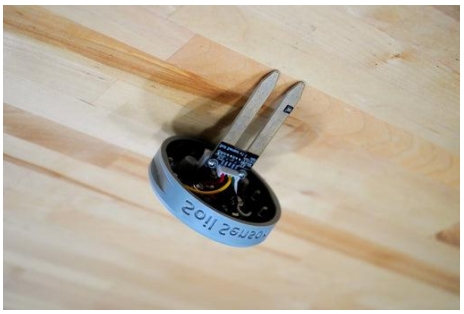


Figure8 : capteur de sol genesis

- **Semoir Genesis(figure)**

Un distributeur de graines précis, ajustable pour différentes tailles de semences. Automatisé et piloté par le logiciel Farmbot, il assure un espacement régulier et une profondeur de plantation cohérente, optimisant la germination.

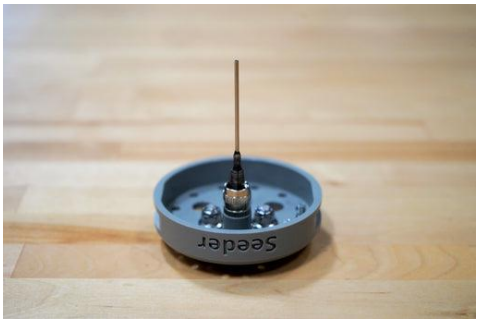


Figure9 : Semoir genesis

Chaque outil est conçu pour s'adapter magnétiquement ou mécaniquement au charriot de farmbot, permettant des échanges rapides sans réglage complexe. Leur compatibilité avec le firmware open-source encourage la personnalisation pour des besoins spécifiques.

2-Software

De nombreux systèmes logiciels contribuent aux fonctionnalités de Farmbot. Le schéma ci-dessous illustre les différents composants et la circulation des informations.

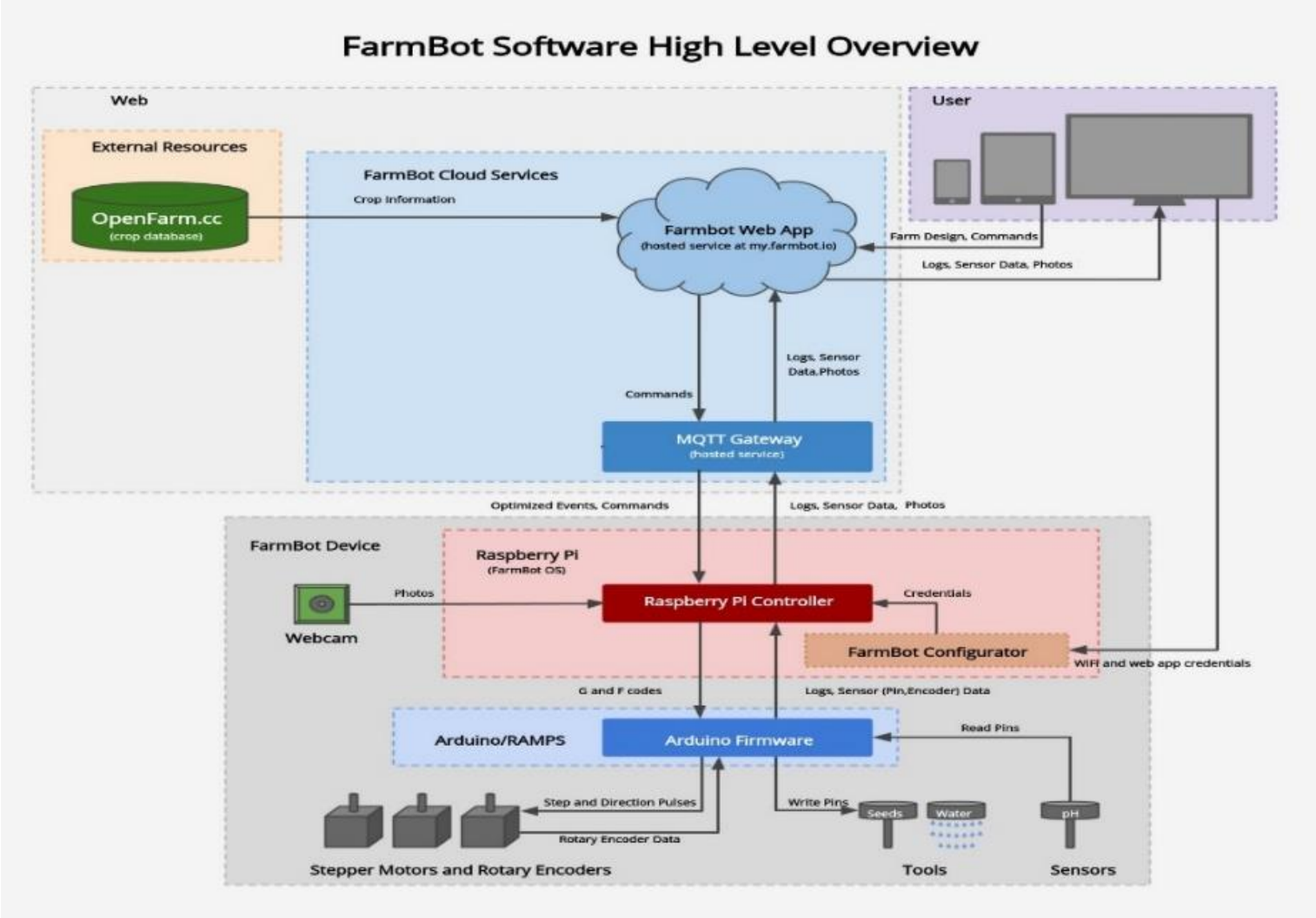


Figure 10 : Architecture du système

Le système FarmBot est un robot potager automatisé, piloté par un ensemble de composants matériels et logiciels qui communiquent entre eux via Internet. Son fonctionnement repose sur trois grandes parties : l'utilisateur, les services en ligne (farmbot cloud services), et le dispositif physique du robot (farmbot device).

L'utilisateur accède à **l'interface web Farmbot (FarmBot Web App)** via un navigateur depuis un ordinateur, une tablette ou un smartphone. Cette plateforme permet de concevoir le plan de culture, de programmer les arrosages, semis, et de surveiller l'état du potager grâce à des retours de données (photos, capteurs, journaux).

L'interface web est connectée à une base de données externe appelée **OpenFarm.cc**, qui fournit des informations sur les plantes. Elle communique aussi avec un **service MQTT**, chargé

de transmettre rapidement et efficacement les commandes et les retours d'information entre le Web App et le robot Farmbot.

Sur le terrain, le FarmBot physique est composé principalement d'un **Raspberry Pi**, qui fait fonctionner le système d'exploitation FarmBot OS. Ce Raspberry Pi reçoit les instructions du Web App via le MQTT, les traite, puis les transmet à une **carte Arduino** (RAMPS) à l'aide du **Raspberry Pi Controller**. L'Arduino, programmé avec un firmware spécifique, pilote ensuite les **moteurs**, les **outils** (comme le **semoir** ou l'**arroseur**), et récupère les grandeurs environnementales avec les **capteurs** (comme le pH ou l'humidité).

Une caméra connectée au Raspberry Pi permet de prendre des photos du potager, qui sont ensuite envoyées à l'application web pour consultation. Toutes les données issues du robot (photos, mesures des capteurs, journaux d'activité) sont également renvoyées vers le Web App, permettant à l'utilisateur de suivre en temps réel l'état du potager.

Enfin, un outil appelé **Farmbot Configurator** permet de configurer les identifiants Wi-Fi et les accès à l'application web lors de l'installation initiale du robot.

Le Raspberry Pi

Le **Raspberry Pi** assure la liaison de communication avec l'application web Farmbot. Il communique directement avec la carte Farmduino. Farmduino est le composant central du Farmbot du point de vue des systèmes électroniques. La figure 12 illustre l'interaction de tous les composants du FarmBot avec le Farmduino, le Raspberry Pi servant de nœud de communication entre l'application web et le Farmduino.

L'application Web FarmBot

L'application web (figure11) nous permet de contrôler et de configurer facilement notre Farmbot depuis un navigateur web, depuis notre ordinateur portable, notre tablette ou notre smartphone. Elle propose des commandes manuelles et un enregistrement en temps réel, un générateur de séquences pour créer des routines personnalisées et les exécuter, ainsi qu'un concepteur de ferme par glisser-déposer pour concevoir graphiquement notre jardin.

Figure 11 : Presentation de l'appli web

Micrologiciel Farmduino

Le **firmware** flashé sur le microcontrôleur Farmduino assure le fonctionnement physique des moteurs, outils, capteurs et autres composants électroniques du farmbot **(figure12)**. Il reçoit les codes du système d'exploitation Farmbot, puis pilote les moteurs, lit et écrit les broches en conséquence. Il renvoie également les données collectées par les encodeurs rotatifs et les lectures des broches au Raspberry Pi.

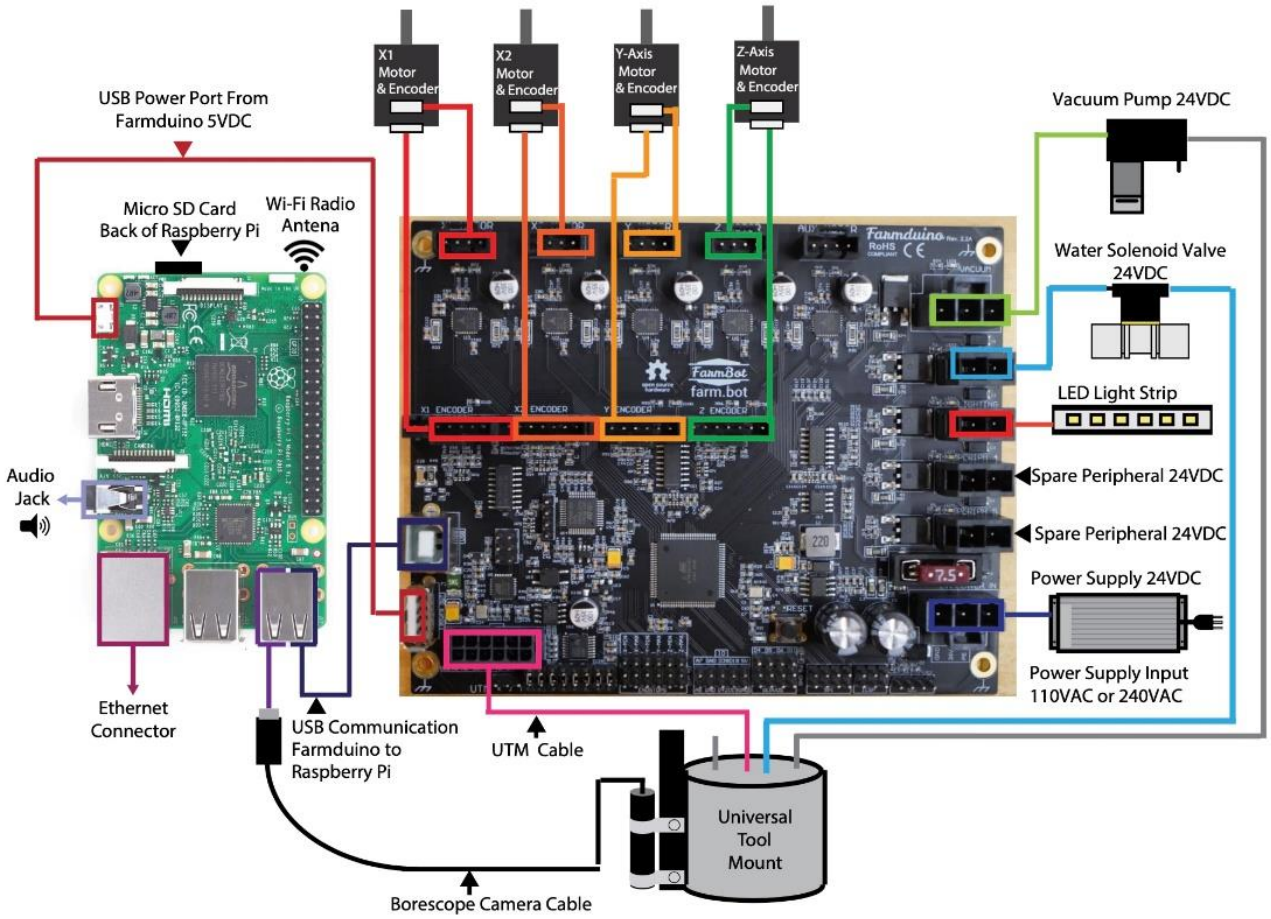


Figure 12 : Architecture du micrologiciel

Configuration du logiciel FarmBot.

- **Étape 1 : Créer un compte d'application Web**
- **Étape 2 : Configuration complète**

1. Après avoir installé FarmBot OS sur la carte SD de l'appareil, on se connecte au farmbot-1234 réseau Wi-Fi depuis notre téléphone portable ou notre ordinateur. Le configurateur ne se chargera pas si nous ne flashons pas la carte SD.

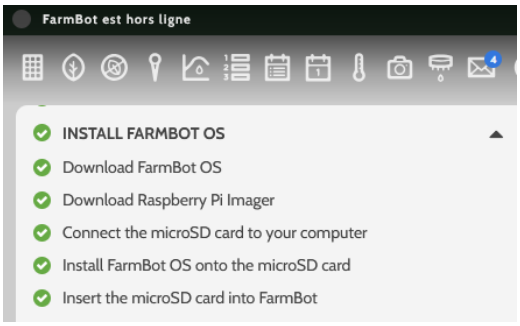


Figure13 : l'installation du Farmbot OS

2. Une fois connecté, un ordinateur portable devrait automatiquement ouvrir un portail captif avec l'utilitaire de configuration. Si nous utilisons un smartphone, nous devons cliquer sur une notification une fois connectée au réseau Farmbot pour ouvrir le portail captif.
3. Sélectionnez WIFI en appuyant sur l'icône correspondante.

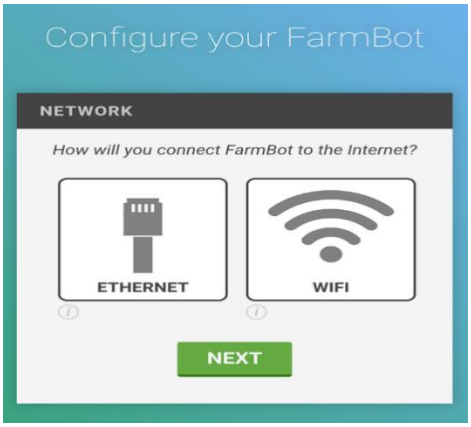


Figure 14 : Configuration du reseau

4. Si nous connectons le Farmbot au Wi-Fi, sélectionnons le NOM DU RÉSEAU Wi-Fi auquel notre FarmBot se connecte normalement (par exemple, mon réseau Wi-Fi REDMI 13 C que j'ai pu partager avec mes données mobiles). Nous pouvons également saisir le nom du réseau Wi-Fi dans le champ « SAISIE MANUELLE ». Appuyez ensuite sur **NEXT.**

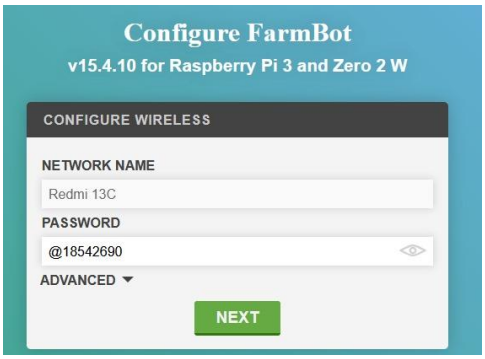


Figure15 : Saisie des paramètres du reseau

5. Saisissez l'adresse e-mail et le mot de passe utilisés lors de la création de notre compte d'application web. Appuyez ensuite sur **FINISH**.

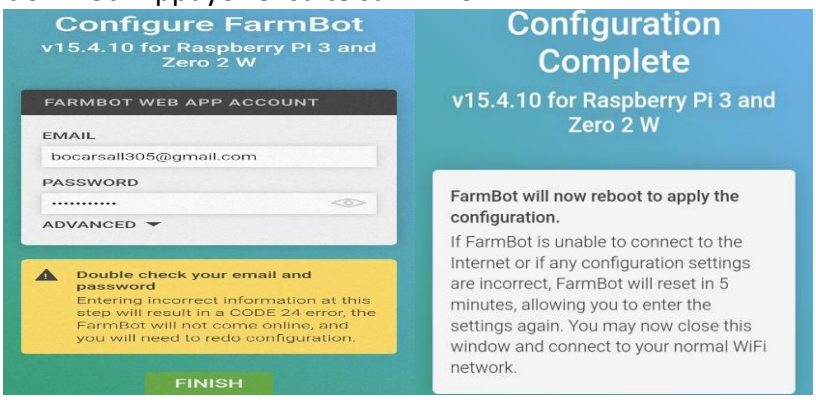


Figure16 : Finalisation de la configuration

Connectivité

Notre FarmBot doit être connecté à Internet et à l'application Web. Lorsque tout est connecté et fonctionne, le **pop-up de connectivité** doit montrer un code **31**. Le code de diagnostic et les points du graphique devraient être verts.

La **fenêtre de connectivité** dans la barre de navigation principale de l'application comporte trois onglets remplis d'informations sur notre Farmbot. Une fois la connexion établie entre Farmbot et l'application web, une icône verte s'affiche à côté de **CONNECTIVITÉ**.

- L'onglet **en temps réel** affiche des informations sur la connexion actuelle de notre ordinateur et de Farmbot à l'application Web, ainsi que certaines mesures pour voir la température du processeur et la tension du Raspberry Pi.

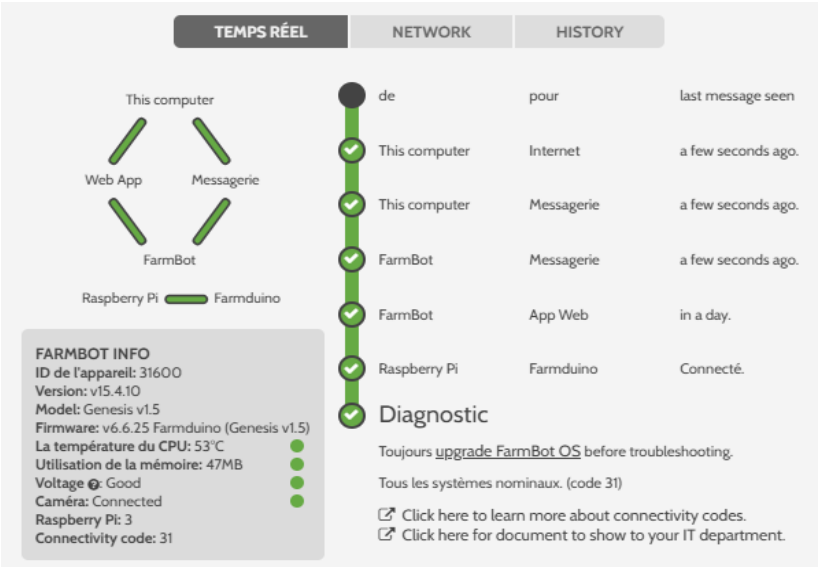


Figure17 : L'onglet en temps réel

Si nous voyons des indicateurs jaunes ou rouges et rencontrons des problèmes avec notre Farmbot, nous devons améliorer la connexion Wi-Fi de notre farmbot.

- L'onglet **réseau** affiche des informations supplémentaires sur la connexion réseau de notre FarmBot, telles que la puissance de son signal Wi-Fi et son adresse IP locale, ainsi que des indicateurs indiquant si les **ports** réseau nécessaires sont ouverts ou non à la fois au Farmbot et à notre ordinateur.

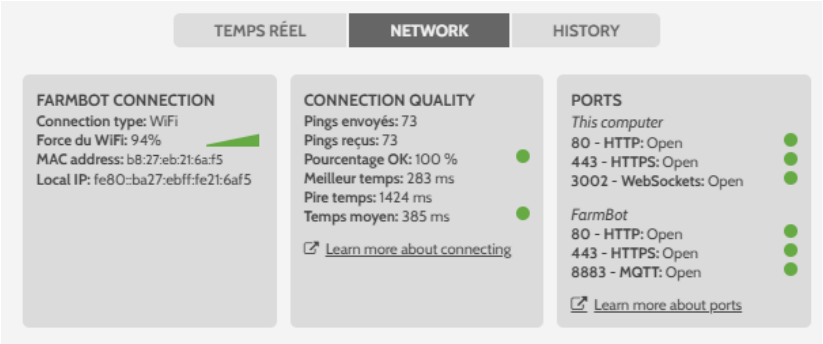
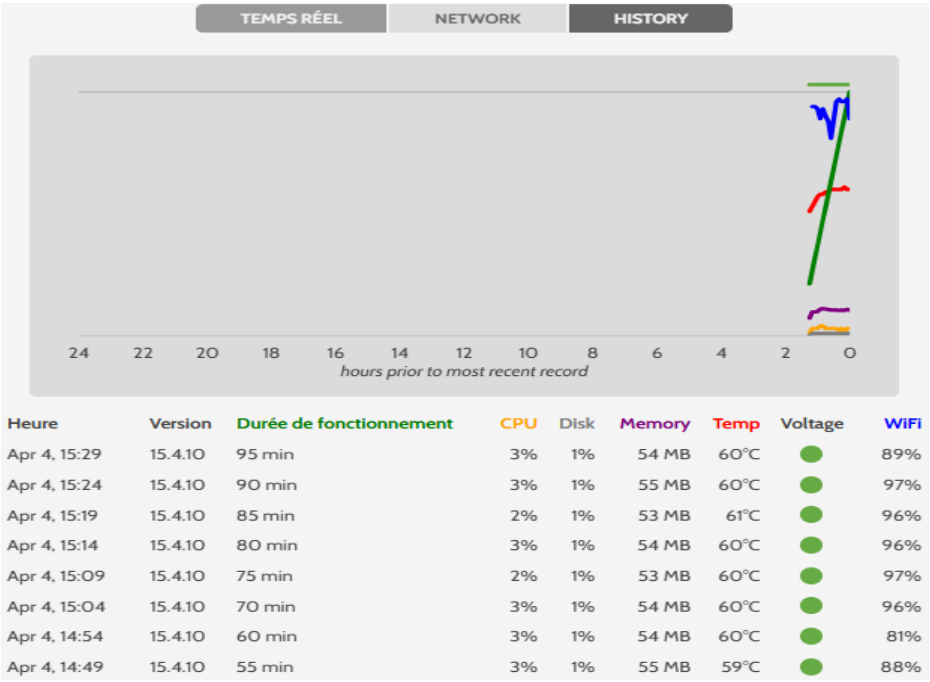


Figure18 : L'onglet reseau

- L'onglet Historique affiche 24 heures d'historique des données. Les données sont collectées toutes les 5 minutes et incluent l'heure, la version du système d'exploitation Farmbot, le temps de fonctionnement (temps écoulé depuis le démarrage de Farmbot), l'utilisation du processeur, du disque dur, de la mémoire, la température du processeur du Raspberry Pi, la tension du système et la puissance du signal Wi-Fi.



Heure	Version	Durée de fonctionnement	CPU	Disk	Memory	Temp	Voltage	WiFi
Apr 4, 15:29	15.4.10	95 min	3%	1%	54 MB	60°C		89%
Apr 4, 15:24	15.4.10	90 min	3%	1%	55 MB	60°C		97%
Apr 4, 15:19	15.4.10	85 min	2%	1%	53 MB	61°C		96%
Apr 4, 15:14	15.4.10	80 min	3%	1%	54 MB	60°C		96%
Apr 4, 15:09	15.4.10	75 min	2%	1%	53 MB	60°C		97%
Apr 4, 15:04	15.4.10	70 min	3%	1%	54 MB	60°C		96%
Apr 4, 14:54	15.4.10	60 min	3%	1%	54 MB	60°C		81%
Apr 4, 14:49	15.4.10	55 min	3%	1%	55 MB	59°C		88%

Figure19 : l'onglet historique

Tâches et les journaux

Les **tâches et les journaux apparaissent** dans la barre de navigation principale de l'application et comportent deux onglets (JOBS et LOGS), ce qui vous permet de suivre les progrès de Farmbot pendant qu'il effectue des travaux de longue durée ainsi que d'auditer les journaux détaillés de chaque opération.

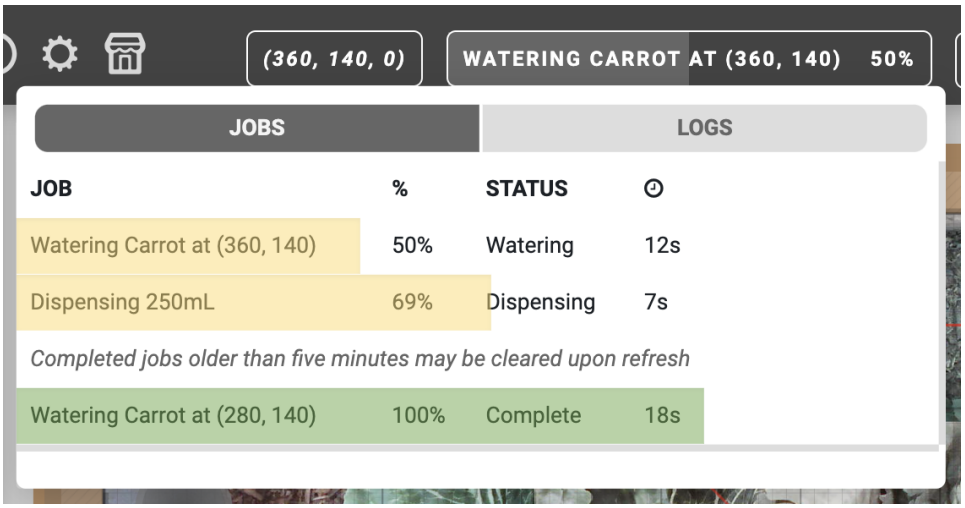


Figure20 : Taches et journaux

Onglet jobs

Lorsque FarmBot exécute des séquences ou des tâches de longue durée telles que l'arrosage de tout le jardin ou la prise de photos, nous pouvons suivre la progression de haut niveau à partir de l'onglet JOBS de la fenêtre contextuelle.

Le tableau des tâches affiche le nom de la tâche, le pourcentage d'achèvement, le statut actuel et la durée de la tâche est en cours d'exécution. Les tâches terminées sont affichées en bas du tableau et peuvent être effacées après 5 minutes d'actualisation de l'onglet du navigateur.

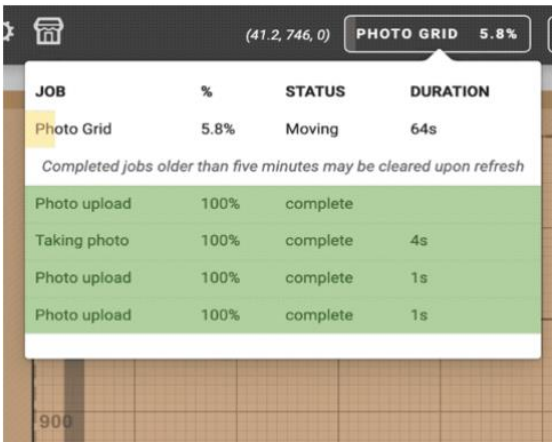


Figure21 : Onglets jobs

Onglet des journaux

L'onglet logs montre une vue de bas niveau de chaque opération individuelle que Farmbot a effectuée. Les logs peuvent être utiles pour le débogage des séquences et le diagnostic des problèmes avec Farmbot, mais ne sont généralement pas nécessaires pendant le fonctionnement normal.

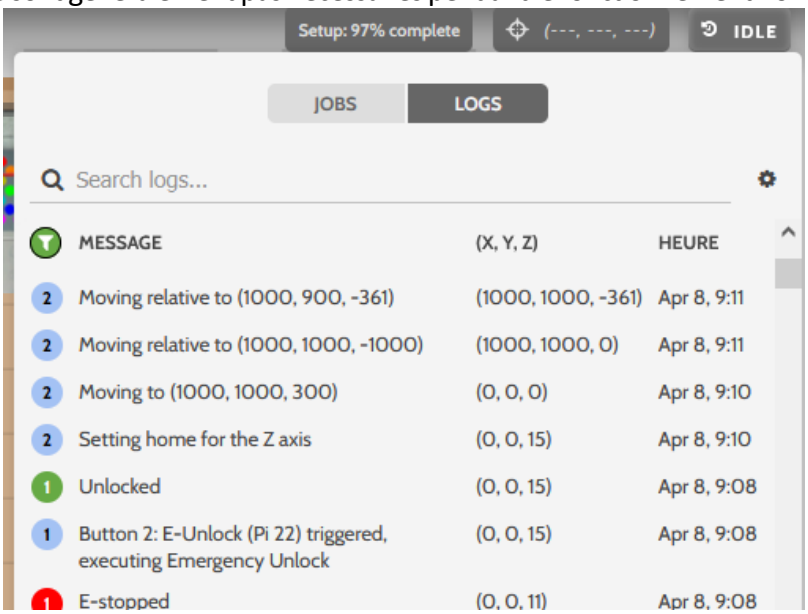


Figure22 : Onglets des journaux

Types de log

Il existe plusieurs **types de log,** chacun avec sa propre couleur, qui indiquent le type de message envoyé.

Type	Signification
SUCCESSUS	FarmBot a mené à bien une tâche. **Exemple:** *Synchronisation*
BUSIE	FarmBot est occupé à travailler sur une tâche. **Exemple:** *synchronisation*
MISE EN GARDE	Une situation peut nécessiter votre attention ou rendre FarmBot insensible. **Exemple:** *verrouillage et échappement d'urgence*
ERROR	Une erreur ou un arrêt d'urgence s'est produit. **Exemple:** *Défaillance du mouvement*
INFO	Informations générales sur ce que fait FarmBot. **Exemple:** *Démarrage de l'eau, de la séquence de toutes les plantes*
FUN	Des journaux qui sont juste pour s'amuser
DEBUG	Informations verbeuses pertinentes pour le développement et le dépannage de logiciels. **Exemple:** *L'interface réseau a besoin d'une configuration: wlan0*
Assaarution	Résultats des commandes ASSERTION (avancées)

Figure23 : Les types de logs

LED du boitier électronique

En plus des LED situées sur le Raspberry Pi, le kits Genesis v1.5 comportes des LED montées sur le dessus du boitier électronique.

Figure24 : Boitier LED sur kit Genesis

LED 1 (synchronisation)

Cette LED **verte** indique **l'état de synchronisation** entre Farmbot et l'application web. Elle est connectée à la broche 24 du GPIO du Raspberry Pi.

Vert (synchronisation)	Statut
(vert uni)	Synchronisé et prêt
(clignotant lentement)	Nécessite une synchronisation (n'exécutera aucun événement ou séquence non synchronisé)
(clignotant rapidement)	Synchronisation
(désactivé)	Hors ligne - Vérifiez le voyant d'état de la connexion

Figure25 : Indication de la LED verte

LED 2 (connexion)

Cette LED bleue indique **l'état de la connexion** entre FarmBot, Internet et l'application web. Elle est connectée à la broche 25 du GPIO du Raspberry Pi.

Bleu (connexion)	**Statut**
(Bleu uni)	Connecté et fonctionnel
(Clignotant lentement)	Nécessite une configuration ou vérifiez que le réseau auquel FarmBot est connecté est en ligne.
(Désactivé)	Hors ligne - Vérifiez votre connexion Internet.

Figure26 : Indication de la LED Bleu

LED 3 et LED 4 (personnalisables)

Ces Leds blanches sont contrôlables par l'utilisateur via la séquence de commandes « Contrôle du périphérique ».

Elles sont respectivement connectées aux broches 12 e 13 du GPIO du Raspberry Pi.

Boutons du boitier électronique

Ce kit comprend également des boutons poussoirs dotés de lumières LED intégrées.

- **Bouton d'arrêt d'urgence**

Le bouton d'arrêt d'urgence est doté d'une LED rouge indiquant si Farmbot est en marche ou non. Il est connecté à la broche 17 du BCM du GPIO du Raspberry Pi.

- **Bouton de déverrouillage**

Le bouton de déverrouillage est doté d'une LED jaune indiquant si Farmbot est activé ou non. Il est connecté à la broche 23 du GPIO du Raspberry Pi.

- **Autres boutons**

Ce kit inclut trois boutons supplémentaires (bouton 3, bouton 4 et bouton 5) dont les actions peuvent être personnalisées via l'interface utilisateur du bouton-poussoir, cependant les LED de ces boutons ne sont pas personnalisables.

3- Dépannage

Lors d'une phase de test du robot potager FarmBot, j'ai rencontré un problème empêchant le déplacement précis du robot sur ses axes X, Y et Z. Le système semblait bloqué ou réagissait de manière aléatoire aux commandes de déplacement envoyées via l'interface de contrôle.

Avec la documentation du farmbot et quelques tutoriaux suivis, j'ai pu faire un diagnostic mécanique et électronique du système. Après avoir vérifié que le logiciel envoyait bien les ordres de mouvement, j'ai décidé d'examiner les éléments matériels.

Les étapes de dépannage réalisées ont été les suivantes :

- **Vérification mécanique des axes** : J'ai commencé par desserrer légèrement les fixations des rails avec les outils du farmbot pour voir si le blocage était dû à un mauvais alignement ou à une contrainte mécanique. Une fois les éléments ajustés, j'ai revissé et réaligné proprement les axes, en m'assurant qu'il n'y avait pas de friction excessive.
- **Contrôle des encodeurs** : Les encodeurs sont essentiels pour assurer la précision du positionnement du robot. Ils permettent de mesurer la rotation des moteurs et donc la distance parcourue. En ouvrant le boitier electronique du farmbot J'ai constaté que

certains fils d'encodeur étaient partiellement mal branches. Je les ai débranchés, puis reconnectés soigneusement, en m'assurant que les connexions étaient bonnes.

Grâce à cette intervention, le FarmBot a pu à nouveau se déplacer correctement sur ses trois axes. Cette expérience m'a permis de mieux comprendre l'importance de l'alignement mécanique et des connexions électroniques dans un système automatisé, ainsi que d'apprendre une méthode de diagnostic pas-à-pas.

Figure27 : Outils mécaniques du farmbot

Chapitre II : Mise en place du système de surveillance des plantes avec Raspberry Pi

INTRODUCTION

Dans le cadre du projet farmbot, j'ai conçu et déployé un système pour surveillance des plantes afin de renforcer la précision et l'efficacité du suivi des cultures. Ce système repose sur l'utilisation d'un Raspberry Pi, un nano-ordinateur compact et économique, capable d'assurer à la fois la capture d'images et le traitement des données localement.

Le choix du **Raspberry Pi** s'est imposé grâce à sa compacité, sa faible consommation énergétique et sa compatibilité avec Python. Il permet à la fois de piloter une caméra et d'exécuter des modèles d'intelligence artificielle.

L'intégration d'une intelligence artificielle (IA) dans ce système répond à un enjeu majeur de l'agriculture moderne : détecter l'apparition de maladies sur les végétaux. En effet, certaines maladies se manifestent visuellement par des anomalies sur les feuilles (taches, décolorations, flétrissures) que l'œil humain pourrait ne pas remarquer immédiatement ou de façon constante. Grâce à l'IA, il devient possible de détecter automatiquement ces signes visuels à partir des images capturées, offrant ainsi un outil d'aide à la décision rapide et fiable pour les utilisateurs du farmbot.

1. Matériel utilisé

- Raspberry Pi 4

Le Raspberry Pi 4 est le cœur de notre système d'analyse. Ce nano-ordinateur gère la capture des images via la caméra et exécute les scripts nécessaires au traitement des données. Avec son processeur multicœur et ses connexions Wi-Fi/Bluetooth, il peut transférer les photos vers un modèle d'IA pour analyse. Sa polyvalence permet aussi d'héberger un tableau de bord local pour visualiser les résultats en temps réel.

Figure28 : Vue de dessus et de cote de la Raspberry

- Caméra Raspberry Pi (IMX219)

La caméra Raspberry Pi équipée du capteur Sony IMX219 capture des images hautes résolutions des plantes, essentielles pour une analyse précise par l'IA. Pour optimiser les résultats, elle peut être fixée sur un support stable et configuré avec un éclairage constant. Des scripts Python automatisent la prise de photos à intervalles réguliers, assurant un suivi cohérent de l'évolution des plantes.

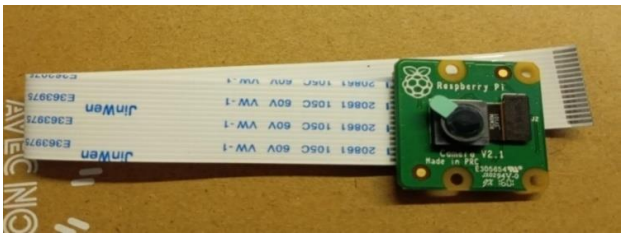


Figure29 : Schéma du connecteur de la caméra Raspberry Pi CSI.

- Carte microSD avec Raspberry Pi OS

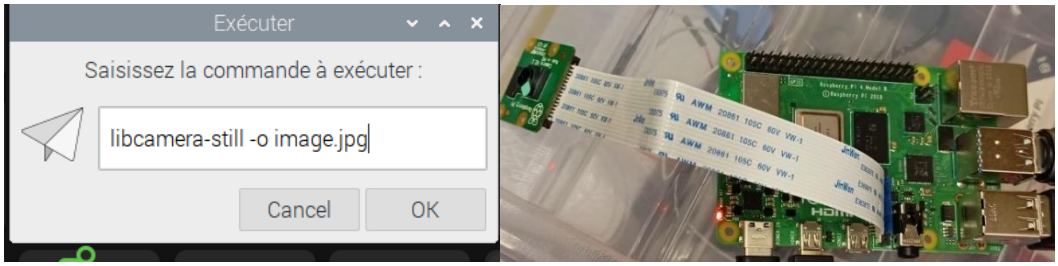
La carte microSD héberge le système d'exploitation Raspberry Pi OS et tous les scripts personnalisés pour la capture et le traitement des images. Elle peut être configurée pour stocker temporairement les images avant leur transfert vers un serveur ou un modèle d'IA, garantissant ainsi une gestion fluide des données.

Spécification de la carte SD

- Les cartes Raspberry Pi SD utilisent la spécification SD6.1 SD.
- Les cartes SD de Raspberry Pi utilisent le facteur de forme microSDHC/microSDXC.
- Les cartes Raspberry Pi SD ont les qualifications suivantes de la classe de vitesse : C10, U3, V30, A2.

2. Mise en place de la caméra

La caméra est connectée au port CSI du Raspberry Pi comme le montre cette video : https://youtu.be/GImeVqHQzsE?t=1 . Après le branchement, la caméra est activée avec l'utilitaire libcamera. Les tests initiaux sont réalisés avec la commande suivante ci-dessous. Vous trouverez la documentation complète de la Raspberry ainsi que tous les autres accessoires dans ce lien : *https://www.raspberrypi.com/documentation/*



- Figure30 : Commande de la capture et connexion de la camera

3. Capture des photos

3.1 Algorithme

Le système commence par créer un dossier de sauvegarde pour les photos en utilisant la fonction os.makedirs(). Il configure ensuite la caméra avec picam2.configure() et la démarre avec picam2.start(). Une fois la caméra prête, une phase de prévisualisation s'effectue pendant 2 secondes grâce à time.sleep(2) afin de stabiliser l'image. Le système passe ensuite à la capture d'image : il génère un nom de fichier unique à l'aide de datetime.now(). strftime(), reconfigure la caméra pour le mode capture avec un nouveau picam2.configure(), puis capture et enregistre l'image en haute résolution au format JPEG avec la fonction picam2.capture_file(). Enfin, le système attend une heure avant de recommencer le cycle à l'aide de time.sleep(3600), assurant ainsi une surveillance continue.

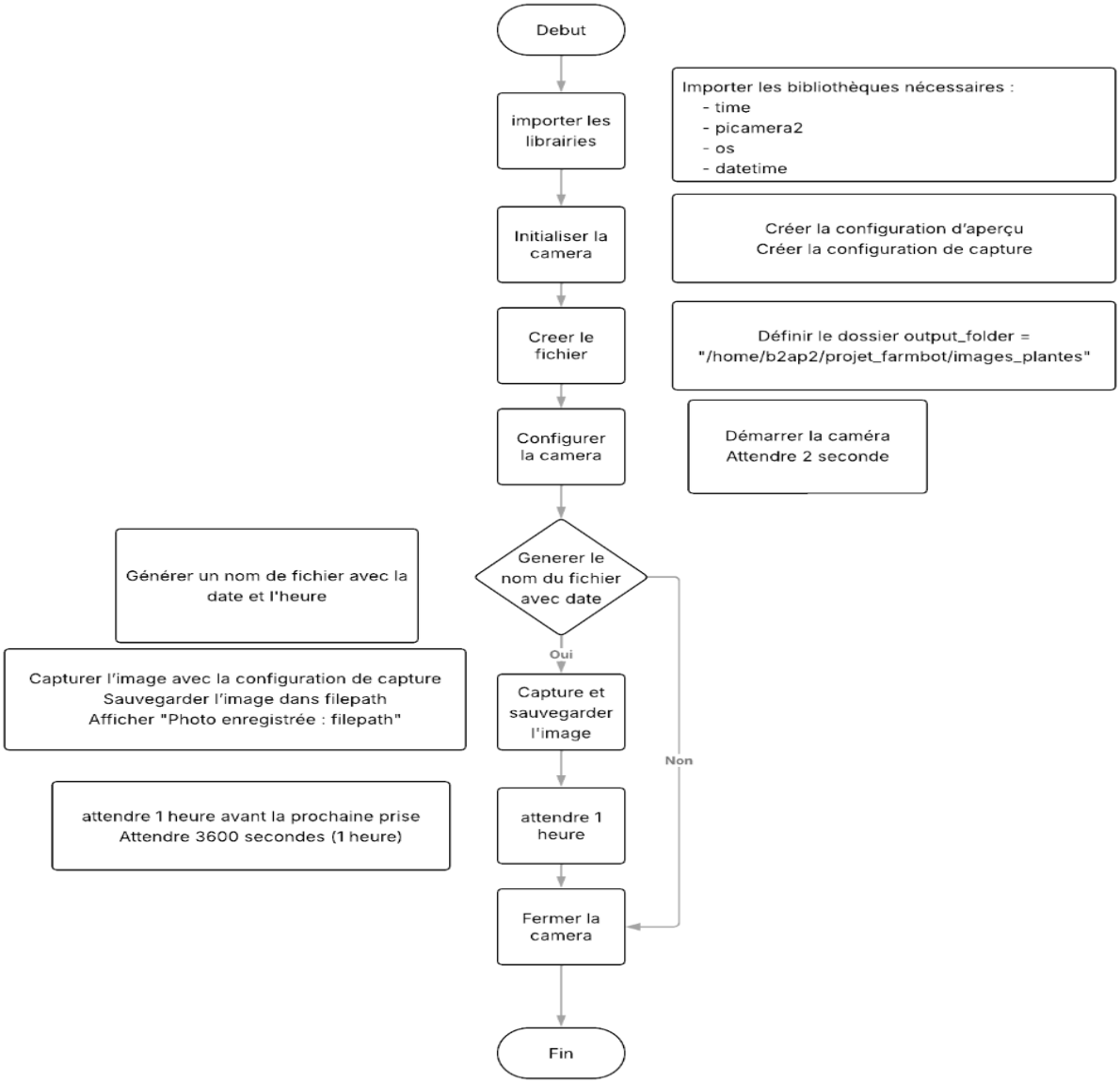


Figure31 : L'algorithme de la capture des photos

3.2 scripte de la capture

Un script Python commenté est lancé en tâche planifiée pour capturer des photos chaque heure.

Vous trouverez ce script via ce lien picamera2/examples/capture_image_full_res.py at main · raspberrypi/picamera2 · GitHub .

```python
import time
from picamera2 import Picamera2, Preview
import os
import datetime

# Initialisation de la caméra
picam2 = Picamera2()
picam2.start_preview(Preview.QTGL)
preview_config = picam2.create_preview_configuration()
capture_config = picam2.create_still_configuration()

# Dossier de sauvegarde des images
output_folder = "/home/b2ap2/projet_farmbot/images_plantes"
os.makedirs(output_folder, exist_ok=True)

# Configuration de la caméra
picam2.configure(preview_config)
picam2.start()
time.sleep(2)
```

Par contre ce code a été légèrement modifier en y ajoutant quelques lignes de code comme suite pour une capture automatique et organisée des images avec la caméra de la Raspberry Pi. Elle prend des photos à intervalles réguliers, les enregistre avec un nom unique basé sur la date/heure.

```python
try:
    while True:
        # Création du nom de fichier avec date et heure
        timestamp = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
        filepath = os.path.join(output_folder, f"photo_{timestamp}.jpg")

        # Capture de l'image
        image = picam2.switch_mode_and_capture_image(capture_config)
        image.save(filepath)
        print(f"Photo enregistrée : {filepath}")

        # Délai d'une heure entre les prises de vue
        time.sleep(3600)

finally:
    picam2.close()
```

Chapitre III : Prédiction d'images à l'aide d'un modèle IA

INTRODUCTION

L'intelligence artificielle (IA) occupe aujourd'hui une place centrale dans de nombreux domaines, notamment en agriculture, où elle permet d'améliorer la précision du suivi des cultures. Dans le cadre de notre projet farmbot, j'ai intégré une IA afin de réaliser automatiquement l'analyse des images capturées par la caméra de la Raspberry. L'objectif est de détecter l'état de santé des plantes.

Le type d'IA utilisé ici repose sur un modèle de classification d'images, entrainé pour reconnaitre deux classes : ***plante saine*** et ***plante malade***. Ce modèle a été créé à l'aide de la plateforme Teachable Machine, un outil accessible qui permet de générer rapidement des modèles d'apprentissage automatique à partir d'un jeu d'images. Une fois le modèle entrainé, il a été converti au format TensorFlow Lite, ce qui le rend compatible avec le Raspberry Pi et optimisé pour une exécution locale, sans nécessiter de connexion internet.

Ce choix technologique permet de disposer d'un système autonome et rapide. Il s'inscrit dans une démarche d'agriculture intelligente, où l'IA devient un outil précieux pour la détection des maladies et la réduction des pertes de production.

1.Téléchargement du modèle IA

Pour détecter l'état de santé des plantes à partir des images capturées par la caméra du de la raspberry pi, j'ai utilisé Teachable Machine, un outil développé par Google permettant de créer des modèles d'intelligence artificielle. Cet outil repose sur le principe de l'apprentissage supervisé, où l'utilisateur fournit un ensemble d'images d'entrainement, afin que le modèle apprenne à reconnaitre des motifs spécifiques.

Dans mon cas, j'ai entrainé un modèle de classification d'images en définissant deux catégories distinctes :

- **Plante malade** : cette classe comprend des images de plantes présentant des signes de maladie, tels que des taches brunes ou noires, un jaunissement des feuilles, un flétrissement.
- **Plante saine** : cette classe regroupe des images de plantes avec un feuillage vert uniforme et une structure normale témoignant d'un bon état de santé.

Source des images utilisées

Pour constituer l'ensemble d'entrainement, j'ai utilisé une base de données publique issue de la plateforme **Mendeley Data**, disponible à l'adresse suivante : https://data.mendeley.com/datasets/hb74ynkjcn/1.

Ce dataset contient des images de feuilles de plantes présentant divers types de maladies ainsi que des exemples de feuilles saines. L'utilisation de ce dataset m'a permis de disposer d'un large éventail de cas visuels pour renforcer la robustesse du modèle.

Intégration du modèle dans le système embarqué

Une fois le modèle entrainé et exporté au format .tflite (compatible avec le Raspberry Pi), je l'ai intégré dans le programme de surveillance des plantes. Les images capturées automatiquement par la caméra de la raspberry sont alors analysées par ce modèle, qui attribue une probabilité de classification à chaque image (**saine** ou **malade**).

Avantages de cette approche

Ce processus permet une surveillance automatisée et continue de l'état des plantes. Grâce à la détection avec l'IA, il devient possible d'intervenir rapidement (traitement, irrigation ciblée, ajustement des paramètres de culture, etc.) avant que la situation ne s'aggrave. Cette approche favorise ainsi une agriculture plus efficace, durable et intelligente, en optimisant les rendements tout en réduisant l'usage excessif d'intrants (engrais, pesticides).

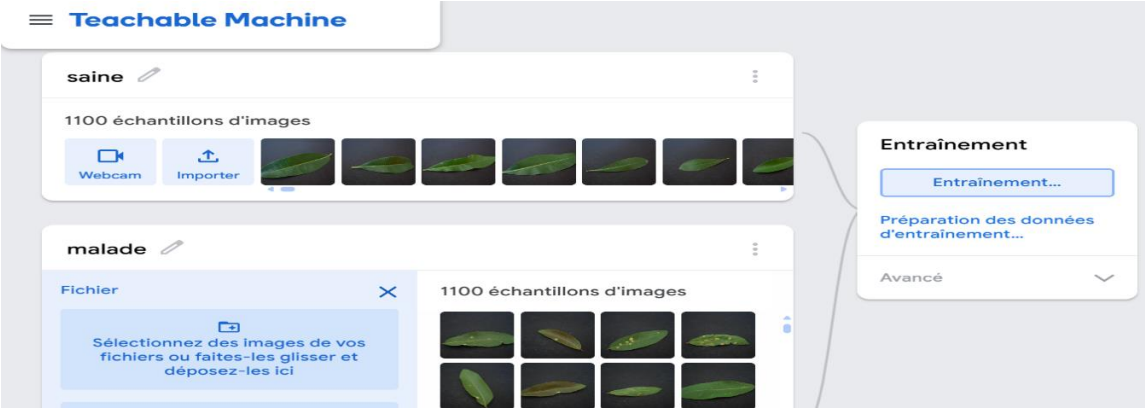


Figure32 : Entrainement du model

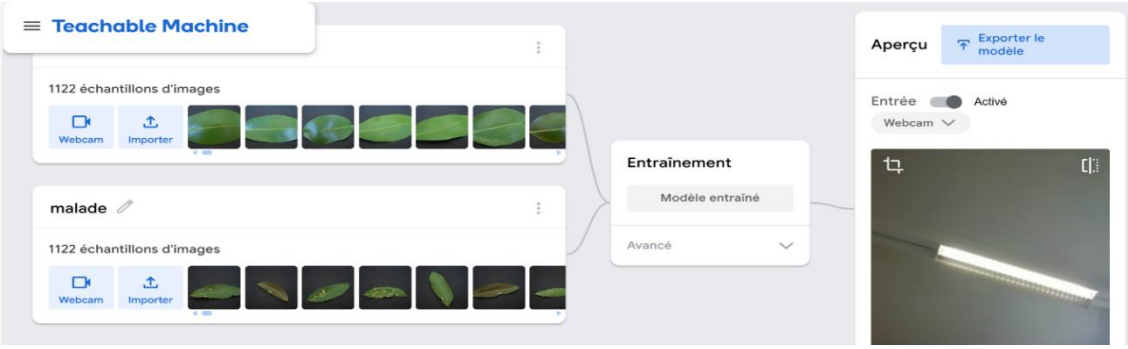


Figure33 : Exportation du model

Après l'entrainement du modèle sur Teachable Machine, j'ai procédé à l'**exportation au format TensorFlow Lite (**Figure33**)**, adapté aux environnements à notre Raspberry Pi. Cette exportation a généré deux fichiers essentiels :

- **model_unquant.tflite** : il s'agit du fichier du modèle d'intelligence artificielle utilisé par le système. Ce fichier contient toutes les informations nécessaires pour que le modèle puisse analyser les images et reconnaitre si une plante est saine ou malade. Il a été compressé pour s'exécuter de manière rapide et efficace sur le Raspberry Pi, même sans grande puissance de calcul
- **labels.txt** : ce fichier contient la liste des classes utilisées pour la classification, ici deux classes ont été utilisées : **malade** et **saine**

Elle permet d'associer l'indice prédit par le modèle à une étiquette compréhensible pour l'utilisateur. Ces fichiers sont ensuite utilisés pour analyser les images et identifier automatiquement l'état des plantes.

2.Installation des bibliothèques nécessaires au traitement

Pour utiliser le modèle IA, plusieurs bibliothèques Python sont nécessaires.

2.1 numpy (version 1.26.4)

Cette bibliothèque est installée via la commande «**pip install numpy==1.26.4 »** essentielle pour le calcul en Python. Elle est utilisée ici pour gérer les matrices d'images (pixels) et effectuer les opérations nécessaires à la préparation des données avant l'analyse par le modèle.

2.2 pillow et tflite-runtime

• **pillow** : bibliothèque de traitement d'images en Python. Elle permet d'ouvrir, convertir les images capturées par la caméra du raspberry dans un format compatible avec le modèle d'IA.

• **tflite-runtime** : cette bibliothèque permet l'exécution de modèles TensorFlow Lite sur le Raspberry Pi. Elle est utilisée pour charger et exécuter le modèle Teachable Machine (format. tflite) afin d'obtenir une prédiction sur l'état de chaque plante.

Elles sont tous les deux installées via la commande « **pip install pillow tflite-runtime »**

3.Implémentation du modèle IA

Le script Python permettant de prédire l'état d'une plante à partir d'une photo est à trouver dans la partie référence.

L'algorigramme

Ce programme est conçu pour analyser automatiquement des images de plantes prises régulièrement par la caméra du Raspberry. Il utilise un modèle d'intelligence artificielle pour détecter des maladies sur les plantes à partir des images. Les résultats de l'analyse sont ensuite enregistrés dans des fichiers .txt.

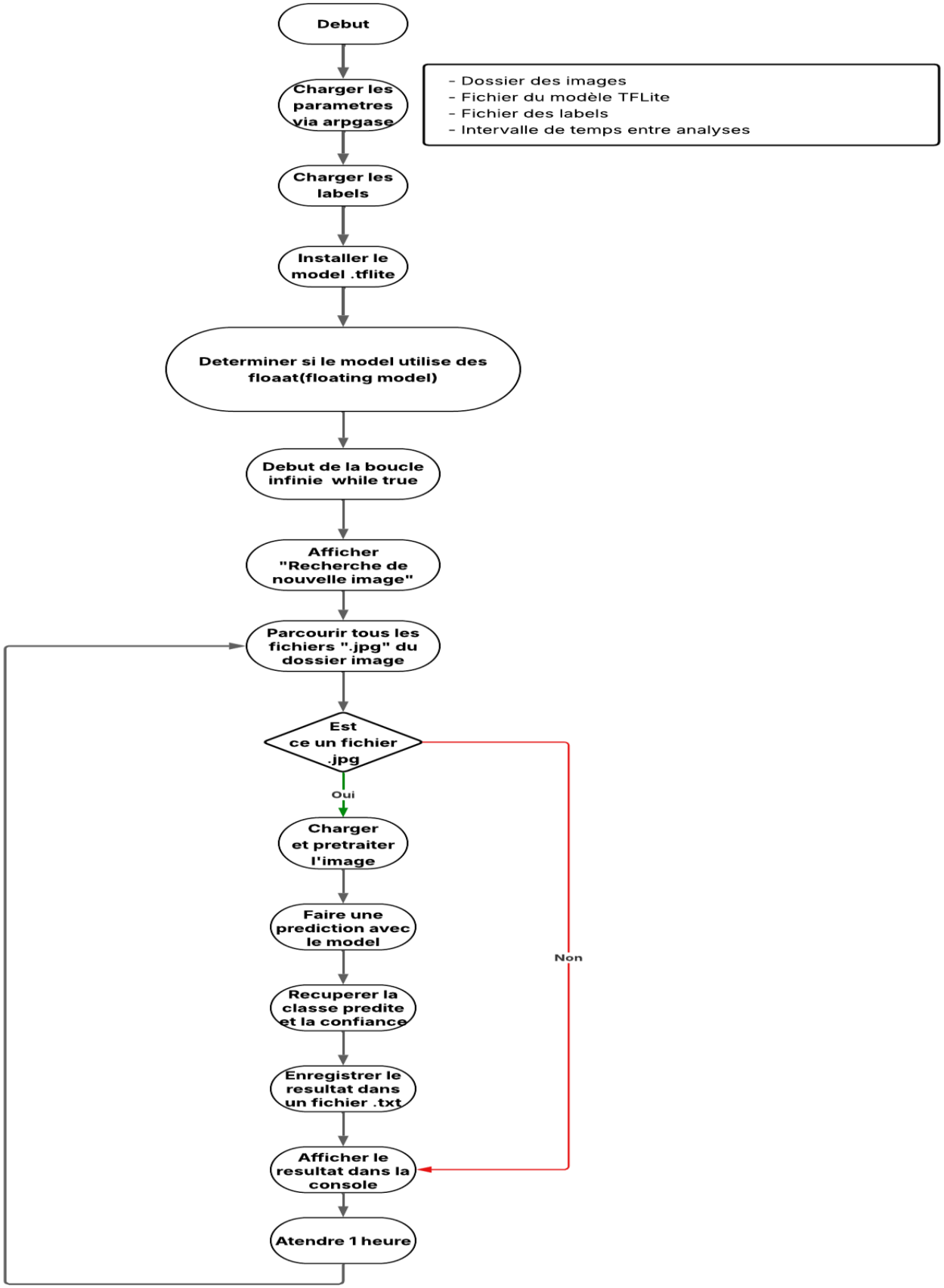


Figure34 : L'algorithme de la prédiction

Résultats obtenus

```
En cours d'exécution: prdt.py
-> 18.jpg → saine (1.00)
-> 19.jpg → malade (1.00)
-> 20.jpg → saine (0.94)
-> 21.jpg → saine (0.91)
-> 22.jpg → malade (0.93)
-> 23.jpg → saine (1.00)
-> 24.jpg → malade (1.00)
```

Figure35 : Console de la prédiction des plantes

Le chiffre entre parenthèse indique le taux de confiance de la prédiction, c'est-à-dire la probabilité estimée par le modèle pour chaque classe. Par exemple, un résultat tel que 1 (1.00) signifie que le modèle estime à 100 % l'exactitude de la classe prédite. une valeur proche de 1 indique une forte certitude, tandis qu'une valeur plus basse (ex 0.65) suggère une prédiction moins fiable, qui peut nécessiter une vérification visuelle.

4. Tableau de bord en ligne (Flask)

Pour faciliter la visualisation et le suivi de l'état des plantes surveillées par la caméra Raspberry Pi, j'ai utilisé un tableau de bord basée sur Flask. Ce tableau de bord permet d'afficher automatiquement les dernières photos prises, triées par ordre chronologique inverse, afin de mettre en évidence l'évolution ou les éventuelles anomalies des feuilles.

En arrière-plan, un script Python parcourt régulièrement le dossier contenant les images. Pour chaque photo au format .jpg, le système vérifie si un fichier .txt de prédiction associée existe (contenant le résultat d'analyse "**sain**" ou "**malade**" avec un score de confiance). Si tel est le cas, le tableau de bord affiche à la fois l'image et son étiquette prédictive. Le tout est présenté de manière simple, avec la date de chaque image, ce qui facilite le suivi dans le temps.

Grâce à cette architecture, chaque fois qu'une nouvelle photo est ajoutée et qu'une prédiction est générée, le tableau de bord se met à jour automatiquement. Cela rend le système particulièrement adapté à une utilisation pour surveiller à distance l'état de santé des plantes

Résultats obtenus

Lorsque les images sont analysées, le tableau de bord affiche par exemple :

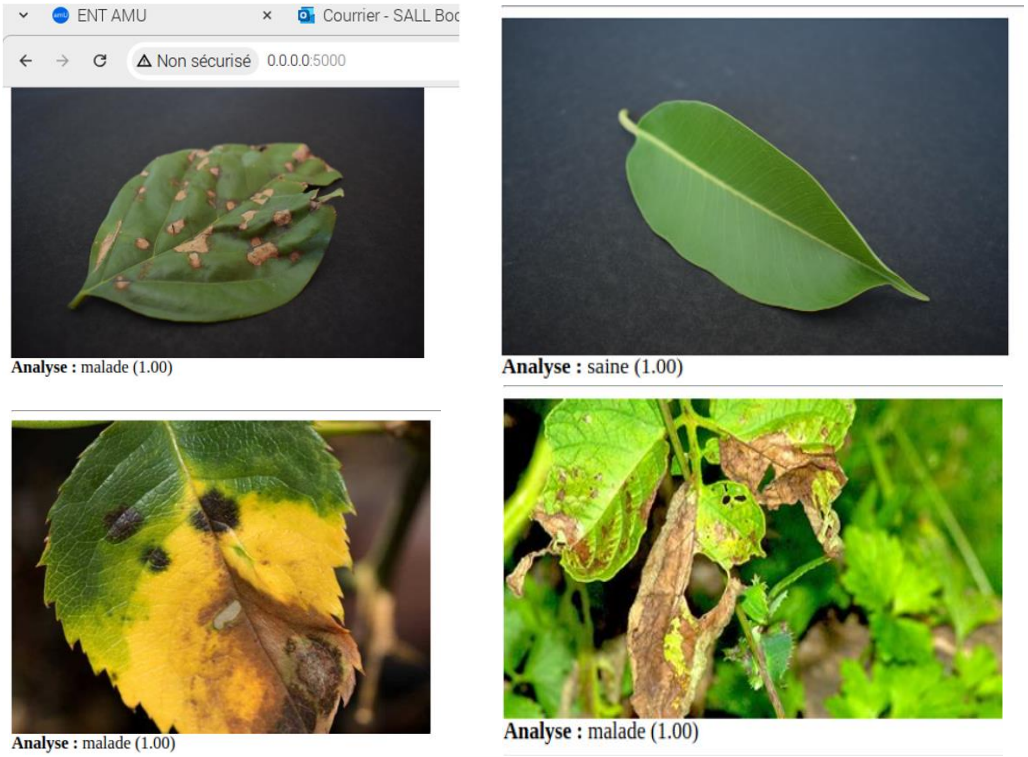


Figure36 : L'affichage des images sur le tableau de bord avec prédiction

Problèmes rencontrés

- **Compatibilité numpy** : Obligation d'utiliser numpy 1.26.4 pour éviter les erreurs de runtime car lors de l'utilisation de bibliothèques d'apprentissage automatique ou de traitement d'image, certaines versions comme **numpy** sont requises pour garantir la compatibilité avec les autres composants du système.
- **Précision limitée** : Une meilleure qualité d'image ou un plus grand nombre d'images d'entrainement permettrait d'améliorer les performances.

Conclusion

Cette intégration d'un modèle IA permet une détection automatique de l'état des plantes, renforçant l'efficacité du projet FarmBot. Il est désormais possible de surveiller la santé des cultures de manière autonome, permettant une réaction rapide en cas d'apparition de maladies.

5.L'algorithme de l'ensemble du Système

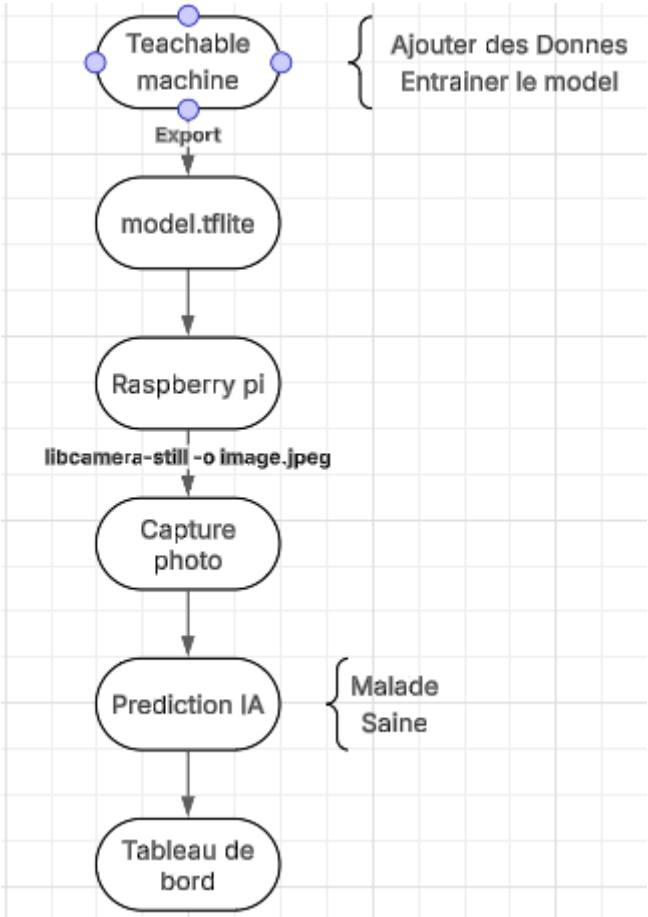


Figure37 : L'algorithme du système de prédiction de l'état des plantes

Chapitre IV : Utiliser Lora Shield et Raspberry Pi pour construire une passerelle LoRaWAN

INTRODUCTION

Le réseau LoRaWAN repose sur une architecture en étoile d'étoiles, où des équipements sans fil basse consommation communiquent avec un serveur applicatif via des passerelles. La modulation utilisée entre équipements et passerelles est LoRa, tandis que les passerelles transmettent les données au serveur via un réseau IP (Ethernet ou cellulaire).

Les équipements ne sont pas directement connectés aux passerelles, qui agissent comme de simples relais. Chaque paquet reçu peut être retransmis par plusieurs passerelles si la couverture le permet, ce qui permet au serveur de dédupliquer les messages et d'effectuer une localisation basée sur les temps d'arrivée.

Lorsqu'une réponse est nécessaire, le serveur choisit la passerelle la plus adaptée en fonction de la qualité du signal. Les objets connectés ne peuvent pas dialoguer directement entre eux; toute communication passe par le serveur applicatif.

1. Le protocole LoRaWAN

1.1 Différence entre LoRa et LoraWAN

Le protocole LoRa est le type de modulation utilisé entre deux Devices LoRa ou entre un Device et une Gateway. Lorsque nous parlons de l'ensemble de la chaine de communication (du Device au serveur LoRaWAN), nous parlons alors du protocole LoRaWAN. LoRaWAN est une extension du protocole LoRa qui permet de connecter de manière sécurisée le Device à un serveur afin de fournir des données à l'utilisateur final.

■ La couche physique LoRa : C'est le type de modulation (Chirp Spread Spectrum) et format de la trame utilisée pour envoyer des données entre un émetteur et un récepteur.

■ Le protocole LoRaWAN : C'est l'architecture complète du réseau (Devices, Gateways, Serveurs) et format de la trame permettant à un Device LoRaWAN de transmettre en toute Sécurité des données à un serveur LoRaWAN.

Comprendre la différence entre le LoRa et le LoRaWAN en vidéo : https://youtu.be/m1-B0jbfqOE.

1.2 Structure du reseau LoRaWAN

La Figure 30 montre l'ensemble de l'architecture LoRaWAN. Sur le côté gauche, on trouve les Devices LoRaWAN qui transmettent les données. L'utilisateur se trouve de l'autre côté et réceptionne les données qui ont transité sur le réseau.

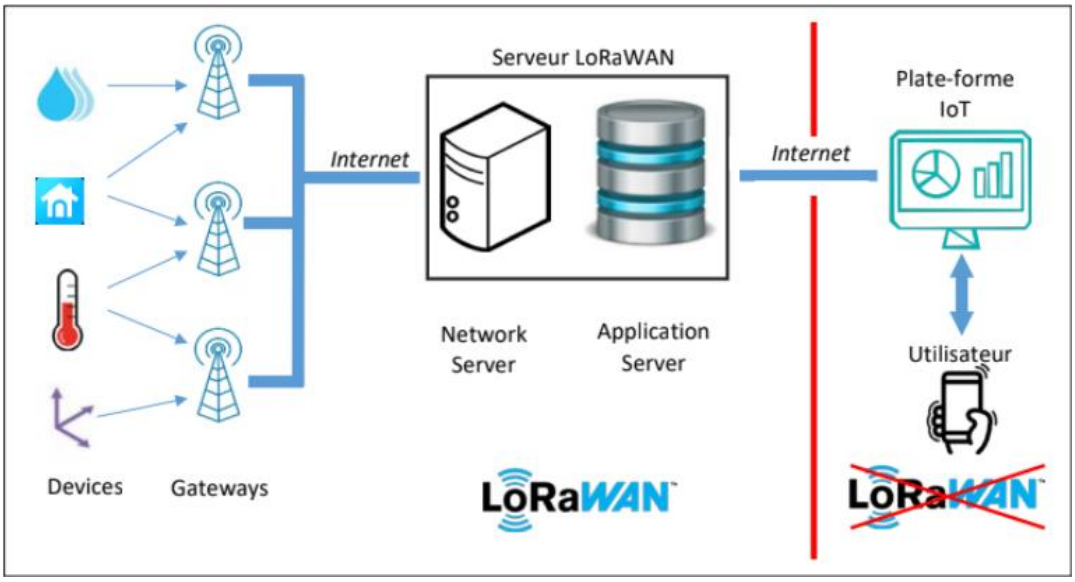


Figure38 : Architecture globale d'un réseau LoRaWAN

Les Devices LoRaWAN, les Gateways, le Network Server et l'Application Server constituent le cœur de l'architecture LoRaWAN, mais la plateforme IoT et la connexion de l'utilisateur (à droite sur la figure) n'ont rien à voir avec ce protocole car il s'agit simplement d'un service web classique.

1.2.1 Les Devices LoRaWAN

Les Devices LoRaWAN sont des systèmes électroniques embarqués appartenant au monde de l'IoT : faible consommation d'énergie, petite taille, faible puissance et faible coût. Ils disposent d'un Transceiver radio LoRa pour atteindre les Gateways. Un Device LoRaWAN ne s'adresse pas spécifiquement à une Gateway car toutes celles présentes dans la zone de couverture reçoivent les messages et les traitent.

1.2.2 Les Gateways LoRaWAN

Elles écoutent sur tous les canaux et sur tous les Spreading Factor en même temps. Lorsqu'une trame LoRa est reçue, elle transmet son contenu par Internet au serveur LoRaWAN réseau qui a été préalablement renseigné dans la Gateway.

D'un côté, la Gateway reçoit un signal modulé en LoRa, et de l'autre côté, elle est connectée à Internet via toutes les connexions possibles : 3G, 4G, 5G, Ethernet, LTE-M...

1.2.3 Le Network Server

Le Network Server reçoit les messages transmis par les Gateways et supprime les doublons (plusieurs Gateways peuvent recevoir le même message et le transmettre au même Network Server). Ensuite, le Network Server authentifie le message grâce à une clé AES 128 bits appelée

NwkSKey (Network Session Key). Nous parlons bien ici d'authentification et non pas de chiffrement.

■ Si le processus d'authentification échoue, le Network Server abandonne le message LoRaWAN.

■ Si le processus d'authentification réussit, le Network Server transfère le message à l'Application Server.

1.2.4 L'Application Server

L'Application Server reçoit des messages chiffrés d'un Network Server. Le chiffrement et le déchiffrement se font grâce à une clé AES 128 bits appelée AppSKey (Application Session Key). L'Application Session Key (AppSKey) est utilisée pour chiffrer les données de l'utilisateur sur le Device LoRaWAN. Les données seront déchiffrées sur l'Application Server. Il s'agit d'un chiffrement symétrique : l'AppSKey du Device doit être la même que celle stockée sur l'Application Server.

1.3 Les différentes classes

1.3.1 Classe A (tous) : Application basse consommation

Tous les Devices LoRaWAN sont de classe A. Chaque Device peut émettre (uplink) vers la Gateway sans vérifier la disponibilité de celle-ci. Cette transmission est suivie de deux fenêtres de réception très courtes. Le Network Serveur (via la Gateway) peut transmettre un message downlink pendant le slot RX1 ou RX2, mais pas les deux.

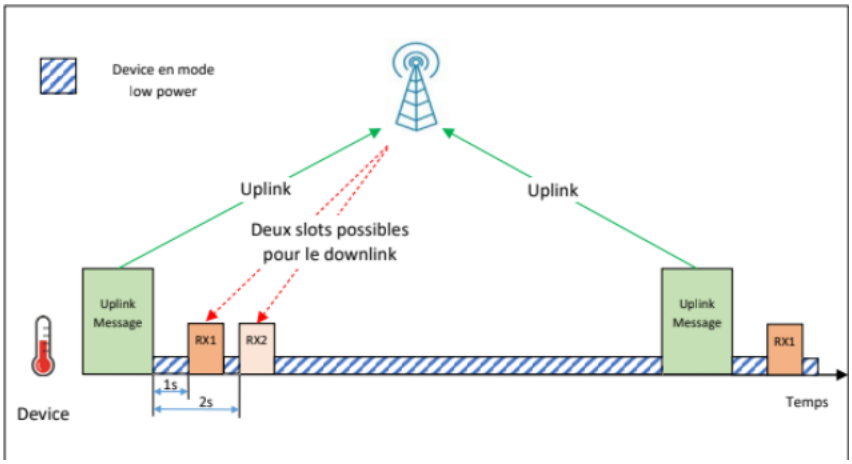


Figure39 : Fenêtres de réception RX1 et RX2 pour un Device de classe A

Un Device de classe A ne peut pas recevoir s'il n'a pas transmis de données en uplink. Par conséquent, vous ne pouvez pas le joindre facilement.

1.3.2 Classe B (Beacon) : Créneau de réception programmé

Les Devices de classe B se comportent de la même manière que les Devices de classe A, mais d'autres fenêtres de réception sont programmées à des moments précis. Afin de synchroniser

les fenêtres de réception des Devices LoRaWAN, les Gateways doivent émettre un signal régulier et périodique appelé balise ou "Beacon".

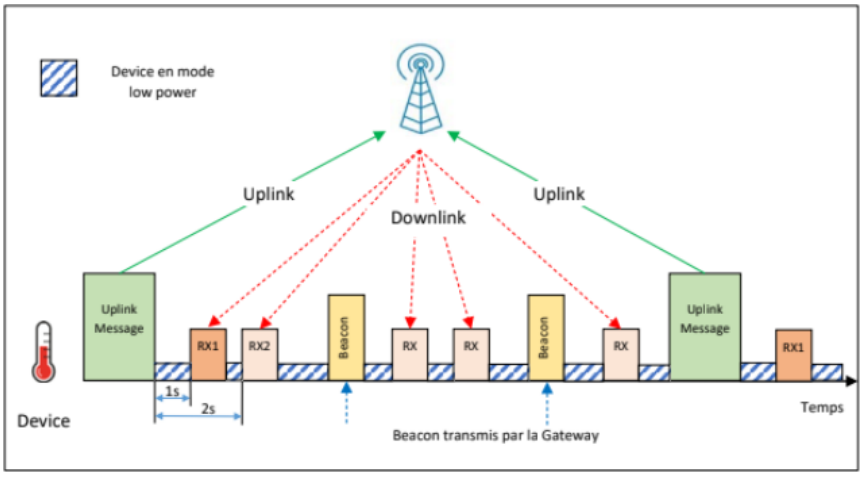


Figure40: Fenêtres de réception pour un Device de classe B

Un Device de classe B peut être joint régulièrement sans devoir nécessairement émettre. En revanche, il consomme plus d'énergie qu'un Device de classe A.

1.3.3 Classe C (Continu) : Écoute en permanence

Les Devices de classe C ont des fenêtres de réception qui sont constamment ouvertes entre deux transmissions. Ces Devices consomment beaucoup plus d'énergie.

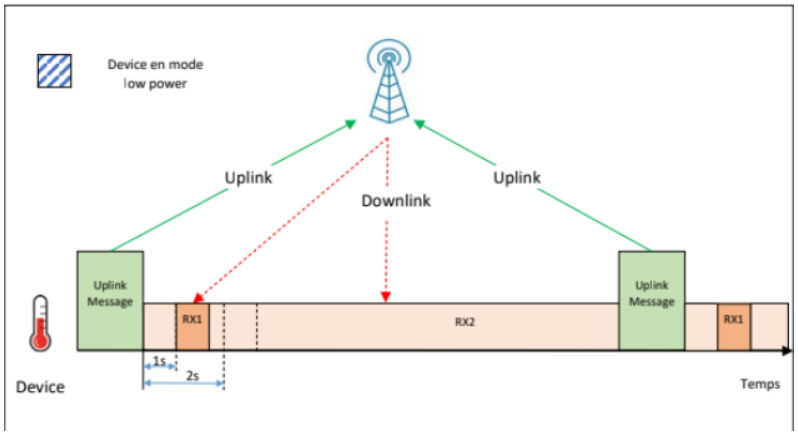


Figure41 : Fenêtres de réception pour un Device de classe C

Le Device LoRaWAN est en écoute permanente entre deux messages uplink. Les paramètres de réception sont fixes (canal, Spreading Factor et bande passante), à l'exception de RX1 qui a toujours le même comportement que dans les classes A et B.

1.4 Activation des devices LoRa ABP ou OTAA

En LoRaWAN, les trois éléments essentiels à la communication sont le DevAddr pour l'identification du Device, ainsi que deux clés : la NwkSKey pour l'authentification et la AppSKey

pour le chiffrement. Deux méthodes sont possibles pour fournir ces informations à la fois sur le Device et sur le serveur LoRaWAN :

- Activation By Personalization : ABP.
- Over The Air Activation : OTAA.

1.4.1 ABP : Activation By Personalization

C'est la méthode la plus simple. C'est donc peut-être celle que nous avons tendance à utiliser pour tester un prototype et établir une communication LoRaWAN.

- Des DevAddr, NwkSKey et AppSKey statiques sont stockés dans le Device.
- Les même DevAddr, NwkSKey et AppSKey sont stockés dans le serveur LoRaWAN.

En ABP, toutes les informations nécessaires à la communication sont déjà connues par le Device, le Network Server et l'Application Server.

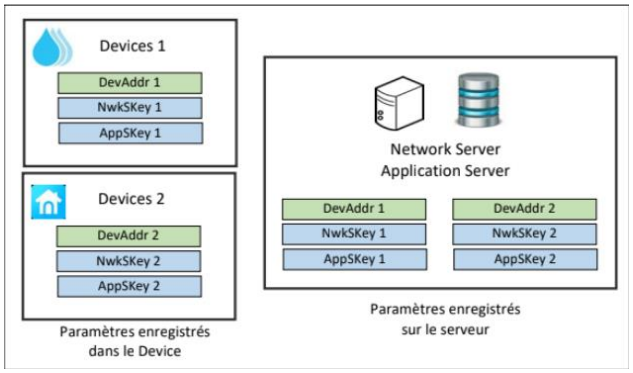


Figure42 : DevAddr, NwkSKey et AppSKey en ABP

1.4.2 OTAA : Over The Air Activation

Avec ce mode d'activation, le DevAddr, l'AppSKey et le NwkSKey seront générés lors d'une procédure de Join. Pour réaliser cette procédure, le Device LoRaWAN doit être configuré avec les éléments suivants :

- DevEUI
- AppEUI/JoinEUI
- AppKey

Le Network Server doit connaître les mêmes DevEUI, AppEUI/JoinEUI, et AppKey et le but principal du Join-Request est de récupérer la configuration finale : DevAddr, NwkSKey et AppSKey des deux côtés.

Tous les éléments nommés "EUI" (Extended Unique Identifier) sont toujours uniques et ont une taille de 8 octets. Les clés sont toujours sur 128 bits (16 octets).

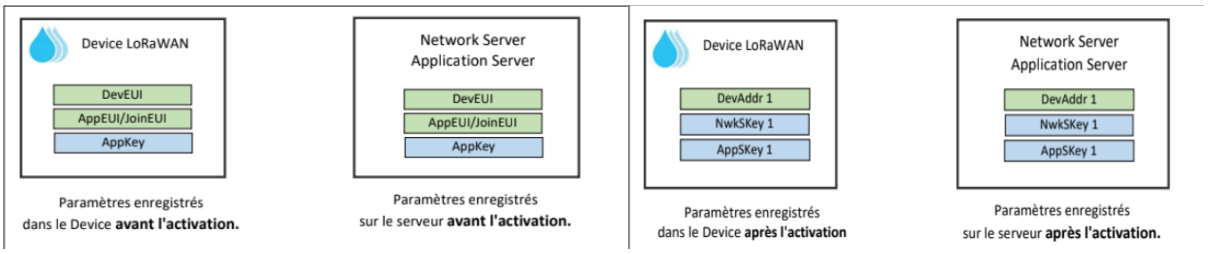


Figure43 : Paramètres stockés avant l'activation OTAA et apres Join request

2. Version de notre reseau LoRaWAN

2.1 Le matériel Utilise

Dans ce projet, j'ai utilisé les deux éléments suivants :

- **Raspberry Pi 4 (Déjà installé)**
- **Dragino Raspberry Pi LoRa/GPS v1.4(Carte fille adaptée à la Raspberry pi 4)**

Le Dragino LoRa/GPS HAT v1.4 est un émetteur-récepteur à longue portée sur un facteur de forme de blindage Arduino et basé sur la bibliothèque Open source. Le module Lora (Dragino) nous permet d'envoyer des données et d'atteindre des gammes extrêmement longues à bas débits de données. Il fournit une communication étalée du spectre à très longue portée et une immunité élevée d'interférence tout en minimisant la consommation de courant.

Pour plus d'informations, veuillez cliquer ici.

Nous utiliserons dans un premier temps les serveurs LoRaWAN et d'application « The Things Network », c'est un réseau de données Internet des objets à base ouverte à l'échelle mondiale. Nous pouvons obtenir plus d'informations sur le site http://thethingsnetwork.org/

Figure44 : Connexion du module dragino et de la raspberry pi

- **Gateway TTIG**

La passerelle **The Things Indoor Gateway (TTIG)** est une Gateway LoRaWAN compacte et facile à déployer, conçue pour une utilisation en intérieur. Elle se connecte à Internet via le

protocole Wi-Fi et permet de relier des objets connectés au réseau LoRaWAN, notamment celui de The Things Network (TTN).

Simple à configurer, elle est idéale pour des projets IoT personnels, éducatifs ou de petites installations professionnelles. TTIG permet la réception des données envoyées par des capteurs LoRa, qu'elle transmet ensuite vers le serveur réseau via le protocole IP. Elle prend en charge la communication bidirectionnelle, ce qui permet également d'envoyer des messages vers les capteurs.

Grâce à sa compatibilité avec TTN, elle permet d'accéder à un réseau mondial collaboratif dédié à l'Internet des objets. Pour plus d'information cliquer ici

Figure45 : Passerelle TTIG

2.2 Mise en place

Noeud de Lora : Le **Raspberry Pi** obtiendra des photos de la caméra et envoyer ces données au module Dragino Raspberry Pi LoRa/GPS v1.4 via l'interface serie SPI.

Portail de Lora : La passerelle **TTIG** recevra ces données et les téléchargera sur le réseau TTN via Internet.

Serveur TTN : Le serveur TTN obtiendra les paquets de données de la passerelle et les données seront stockées à l'endroit correspondant, de sorte que l'on puisse voir les informations envoyées.

Figure : Algorithme du système

2.3 Cree un compte sur le reseau TTN

Avant de démarrer ce projet, il est nécessaire de créer un compte sur **The Things Network (TTN)**. L'inscription peut se faire directement sur le site officiel à cette adresse : https://www.thethingsnetwork.org.

Pour enregistrer nos dispositifs sur TTN, l'options opte est : **Utiliser l'interface graphique** accessible depuis le site de TTN. C'est la méthode la plus simple et la plus intuitive.

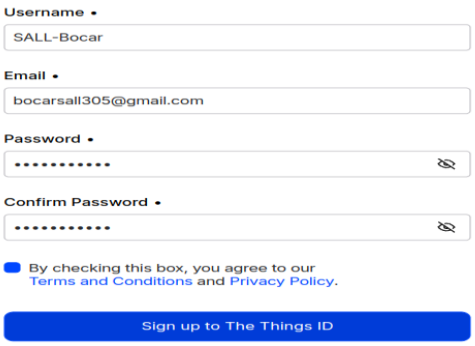


Figure47 : Création d'un compte sur TTN

2.4 Créer une passerelle sur le reseau TTN

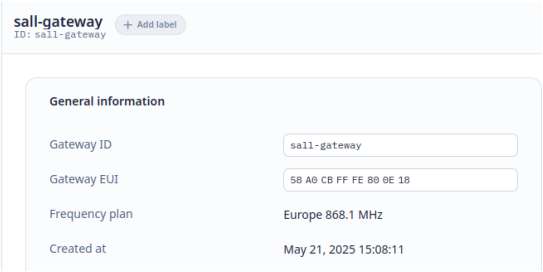


Figure48 : Enregistrement de la passerelle sur TTN

D'abord on se Connecte au réseau TTN par notre propre compte et passer à cette page : https://eu1.cloud.thethings.network/console Cliquez sur **ajouter une passerelle** et nous allons entrer une page pour taper notre Gateway ID . Ensuite, tapez l'identifiant de la passerelle à la colonne Gateway EUI (En hexadécimal).

Nous pouvons voir que le statut de la passerelle est connecté, cela signifie que nous venons de créer une passerelle pour notre compte TTN.

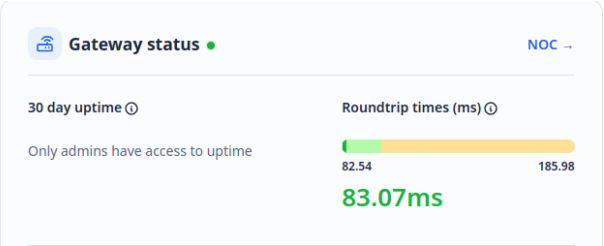


Figure49 : Status de la passerelle

2.5 Ajouter le End device sur le TTN

2.5.1 Communication du Dragino et le Raspberry

Pour ce faire je me suis assure que la bibliothèque d'accès GPIO « wiringpi » est installée en ouvrant le terminal et tapant la commande **sudo apt-get install wiringpi** ensuite j'ai active le SPI en suivant ses étapes.

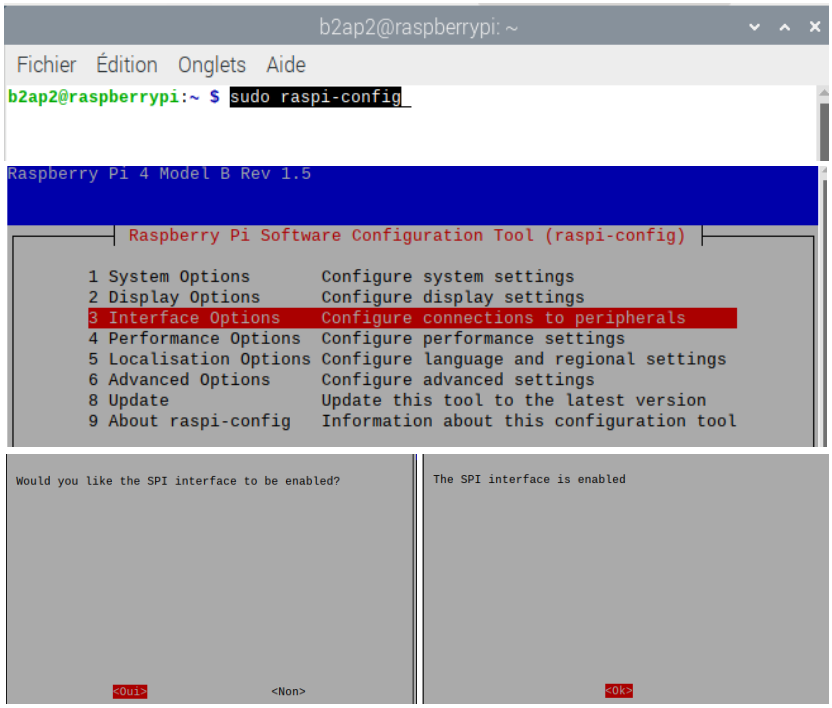


Figure50 : Activation de l'interface serie SPI de la raspberry pi

Apres cela on peut voir que Raspberry Pi a bien le SPI activé et prêt à communiquer avec deux périphériques SPI via /dev/spidev0.0 et /dev/spidev0.1

2.5.2 Récupération du JoinEUI du module dragino

L'objectif de cette étape est d'**obtenir le JoinEUI** du module **Dragino LoRaWAN**, nécessaire pour l'enregistrement du nœud sur un serveur LoRaWAN (comme The Things Network).

Étapes réalisées :

2.5.2.1 Création d'un environnement de travail

J'ai commencé par créer un répertoire spécifique pour isoler ce projet de passerelle LoRa :

Cela permet de garder une organisation claire des fichiers liés à la configuration de la passerelle.

2.5.2.2 Téléchargement d'un forwarder LoRa monocanal

Afin de recevoir les paquets LoRa envoyés par le module Dragino, j'ai téléchargé depuis GitHub un projet open source de **LoRaWAN single-channel packet forwarder** :

```
b2ap2@raspberrypi:~/lorawan_gateway $ git clone https://github.com/tftelkamp/sin
gle_chan_pkt_fwd.git
Clonage dans 'single_chan_pkt_fwd'...
remote: Enumerating objects: 18, done.
remote: Total 18 (delta 0), reused 0 (delta 0), pack-reused 18 (from 1)
Réception d'objets: 100% (18/18), 12.60 Kio | 921.00 Kio/s, fait.
Résolution des deltas: 100% (7/7), fait.
```

Après avoir cloné le dépôt GitHub du **single_chan_pkt_fwd**, je suis allé dans le répertoire avec la commande « cd single_chan_pkt_fwd » puis éditer le fichier main.cpp pour adapter la configuration matérielle et réseau à mon module Dragino et à ma passerelle LoRa.

```
/*******************************************************************************
 *
 * Configure these values!
 *
 *******************************************************************************/

// SX1272 - Raspberry connections
int ssPin = 8;
int dio0  = 4;
int RST   = 25;

// Set spreading factor (SF7 - SF12)
sf_t sf = SF7;

// Set center frequency
uint32_t  freq = 868100000; // in Mhz! (868.1)

// Set locationx
float lat=52.4518;
float lon=4.8119;
int   alt=10.0;

/* Informal status fields */
static char platform[24]    = "Single Channel Gateway";  /* platform definition */
static char email[40]       = "bocarsall305@gmail.com";  /* used for contact email */
static char description[64] = "LoRaWAN single channel avec dragino v1.4";/* used for free form description */

// define servers
// TODO: use host names and dns
#define SERVER1 "54.72.145.119"    // The Things Network: croft.thethings.girovito.nl
//#define SERVER2 "192.168.1.10"      // local
#define PORT 1700                   // The port on which to send data
```

Figure51 : Configuration du fichier main.cpp

Paramètres matériels modifiés :

- **ssPin=6**
  ssPin correspond au "Slave Select » du module LoRa. Ce pin est utilisé pour sélectionner le module LoRa lors de la communication SPI avec le Raspberry Pi. Le pin 6 est utilisé ici car c'est celui qui est relié au module LoRa pour cette fonction.

- **dio0=7**
  dio0 est une des broches d'interruption du module LoRa (souvent utilisée pour indiquer qu'un paquet est reçu). Elle est ici connectée à la broche 7 du Raspberry Pi.

- **RST=0**
  RST est la broche de réinitialisation du module LoRa. Elle permet de redémarrer le

module en cas de besoin. Ici le mettant à 0 signifie qu'aucune broche n'est connectée à cette fonction.

2.5.2.3 Compilation du forwarder et connexion du matériel

Une fois les paramètres adaptés, j'ai compilé le projet avec la commande « **make** ». Cela a généré un exécutable nommé single_chan_pkt_fwd, qui permet de lancer le forwarder.

Avant de démarrer la passerelle, j'ai procédé à la préparation matérielle. J'ai d'abord éteint proprement le Raspberry Pi, puis j'ai coupé son alimentation. Ensuite, j'ai connecté l'antenne au module Dragino.

Après avoir connecté l'antenne, j'ai redémarré le Raspberry Pi, puis lancé le forwarder avec la commande suivante : « **sudo ./single_chan_pkt_fwd** »

```
b2ap2@raspberrypi:~/lorawan_gateway/single_chan_pkt_fwd $ sudo ./single_chan_pkt_fwd
SX1276 detected, starting.
Gateway ID: d8:3a:dd:ff:ff:30:8a:51
Listening at SF7 on 868.000000 Mhz.
------------------
stat update: {"stat":{"time":"2025-05-21 09:42:28 GMT","lati":52.45180,"long":4.81190,"alti":10,"rxnb":0,"rxok"
"rxfw":0,"ackr":0.0,"dwnb":0,"txnb":0,"pfrm":"Single Channel Gateway","mail":"bocarsall305@gmail.com","desc":"L
WAN single channel avec dragino v1.4"}}
```

À ce moment on peut voir les trames dans le terminal et repérer le paquet contenant le **JoinEUI** du module dragino là où est indiqué **Gateway ID**.

Malheureusement à ce stade nous n'avons pas le **Deveui** et **Appskeys** ce qui fais que nous allons configurer nos propres paramètres car celles-ci sont necessaire pour l'ajout du End-device.

Pour recuperer les deux clés j'ai suivi la procédure suivante :

- Exécuter les commandes suivantes pour installer les paquets requis et modifier le fichier /etc/default/gpsd.

« sudo apt install git device-tree-compiler git python3-crypto python3-nmea2 python3-rpi.gpio python3-serial python3-spidev python3-configobj gpsd libgps-dev gpsd-clients python3-pip
pip3 install simplecayennelpp
git clone https://github.com/bbaranoff/libgps
cd libgps
make
sudo make install
sudo ldconfig »

```
Fichier  Édition  Onglets  Aide
  GNU nano 5.4                    /etc/default/gpsd *
# Default settings for the gpsd init script and the hotplug wrapper.

# Start the gpsd daemon automatically at boot time
START_DAEMON="true"

# Use USB hotplugging to add new USB devices automatically to the daemon
USBAUTO="false"

# Devices gpsd should collect to at boot time.
# They need to be read/writeable, either by user gpsd or the group dialout.
DEVICES="/dev/ttyAMA0"

# Other options you want to pass to gpsd
GPSD_OPTIONS="-n"
```

Figure52 : Modification du fichier gpsd

Parmi les paquets installés, figurait le service **gpsd** ainsi que ses dépendances (libgps-dev, gpsd-clients). Bien que ce composant soit normalement utilisé pour des projets intégrant un module GPS mais son absence entrainera des erreurs lors de l'exécution du script test.py, car le scripte importe des modules liés à la géolocalisation (comme libgps, nmea2…), ce qui rend gpsd indirectement nécessaire pour éviter les erreurs d'exécution.

- Ajouter les lignes suivantes dans le fichier **/boot/config.txt** :

```
enable_uart=1
dtoverlay=miniuart-bt
dtoverlay=spi-gpio-cs
```

- Modifier le fichier de Configuration du **/boot/cmdline.txt**

```
1  dwc_otg.lpm_enable=0 console=tty1 root=/dev/mmcblk0p2 rootfstype=ext4 elevator=deadline fsck.repair=yes rootwait
2
```

- Sur le fichier /home/b2ap2/dragino/dragino.ini.default et on le réécrit sous /home/b2ap2/dragino/dragino.ini comme suite ainsi on programme nos propres paramètres OTAA (Deveui et Appskey) ensuite exécuter le script **test.py**

```
labels.txt ✖   main.cpp ✖   config.txt ✖   cmdline.txt ✖   dragino.py ✖   dragino.ini ✖
1   gps_baud_rate = 9600
2   gps_serial_port = /dev/ttyS0
3   gps_serial_timeout = 1
4   gps_wait_period = 10
5
6   #LoRaWAN configuration
7   spreading_factor = 7
8   max_power = 0x0F
9   output_power = 0x0E
10  sync_word = 0x34
11  rx_crc = True
12  #Where to store the frame count
13  fcount_filename = .lora_fcount
14
15  ##Valid auth modes are ABP or OTAA
16  ##All values are hex arrays eg devaddr = 0x01, 0x02, 0x03, 0x04
17  #auth_mode = "abp"
18  #devaddr =
19  #nwskey =
20  #appskey =
21
22  auth_mode = otaa
23  deveui = 0x70, 0xB3, 0xD5, 0x7E, 0xD0, 0x07, 0x0F, 0x0E
24  appeui = 0xD8, 0x3A, 0xDD, 0xFF, 0xFF, 0x30, 0x8A, 0x51
25  appkey = 0x35, 0xCB, 0xFB, 0x23, 0x92, 0x22, 0xF5, 0x90, 0x84, 0x5E, 0xE4, 0x64, 0x55, 0x99, 0x47, 0xAA
```

Figure53 : Configuration des paramètres OTAA du module dragino

Une fois toutes ces étapes terminées, on peut se rendre sur The Things Network (TTN) pour ajouter notre module Dragino en tant qu'End Device, de la même manière que nous avons enregistré la passerelle, mais cette fois en utilisant les paramètres OTAA que nous avons configurés précédemment.

```
1   """
2       Test simple pour le module Dragino - envoie un payload brut en LoRaWAN
3   """
4   import logging
5   from datetime import datetime
6   from time import sleep
7   import RPi.GPIO as GPIO
8   from dragino import Dragino
9   import binascii
10  # Configuration du niveau de log
11  logLevel = logging.DEBUG
12  logging.basicConfig(
13      filename="test.log",
14      format='%(asctime)s - %(funcName)s - %(lineno)d - %(levelname)s - %(message)s',
15      level=logLevel
16  )
17  # Initialisation du module Dragino avec le fichier de config
18  D = Dragino("/home/b2ap2/dragino/dragino.ini", logging_level=logLevel)
19  # Démarrage de la procédure OTAA (join)
20  D.join()
21  while not D.registered():
22      print("Waiting for JOIN ACCEPT")
23      sleep(10)
24  # Payload brut à envoyer (remplace ceci par le payload que tu veux envoyer)
25  hex_payload = "48656c6c6f576f726c64"  # Correspond à "Hello"
26  raw_payload = list(binascii.unhexlify(hex_payload))
27  # Envoi du payload
28  D.send_bytes(raw_payload)
29  start = datetime.utcnow()
30  while D.transmitting:
31      pass
32  end = datetime.utcnow()
33  print(f"Payload envoyé en {end - start}")
```

Figure54 : Script de test pour l'envoie du dragino

Apres l'exécution on a le dragino qui attend le **JOIN ACCEPT** de la passerelle TTIG afin de lui envoyer des données.

```
● b2ap2@raspberrypi:~/dragino $ /bin/python /home/b2ap2/dragino/test.py
  /home/b2ap2/dragino/dragino/SX127x/board_config.py:52: RuntimeWarning: This channel is already in use, continuing
   anyway.  Use GPIO.setwarnings(False) to disable warnings.
    GPIO.setup(BOARD.LED, GPIO.OUT)
  Mode <- SLEEP
  Mode <- FSK_STDBY
  Mode <- SLEEP
  Mode <- FSK_STDBY
  Mode <- SLEEP
  Mode <- STDBY
  Mode <- TX
  Waiting for JOIN ACCEPT
  Mode <- STDBY
  Mode <- RXCONT
  Waiting for JOIN ACCEPT
  Waiting for JOIN ACCEPT
  Mode <- SLEEP
  Mode <- STDBY
  Mode <- TX
  Mode <- STDBY
  Payload envoyé en 0:00:00.067477
  Mode <- RXCONT
```

Figure55 : Console du scripte d'envoie du module dragino sur la passerelle

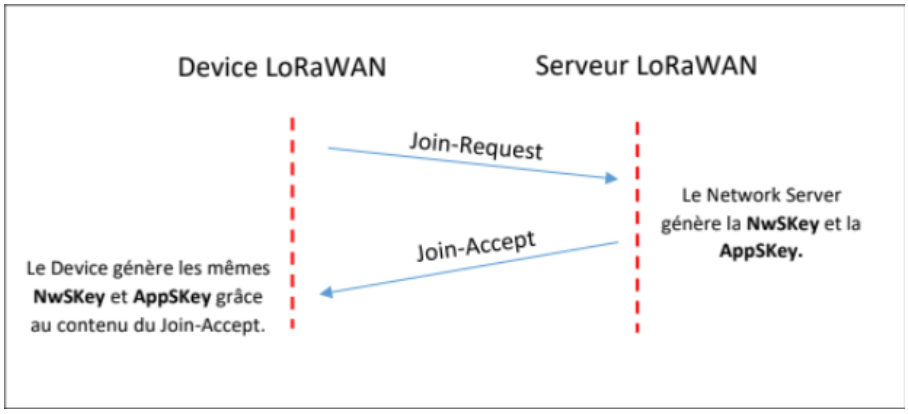
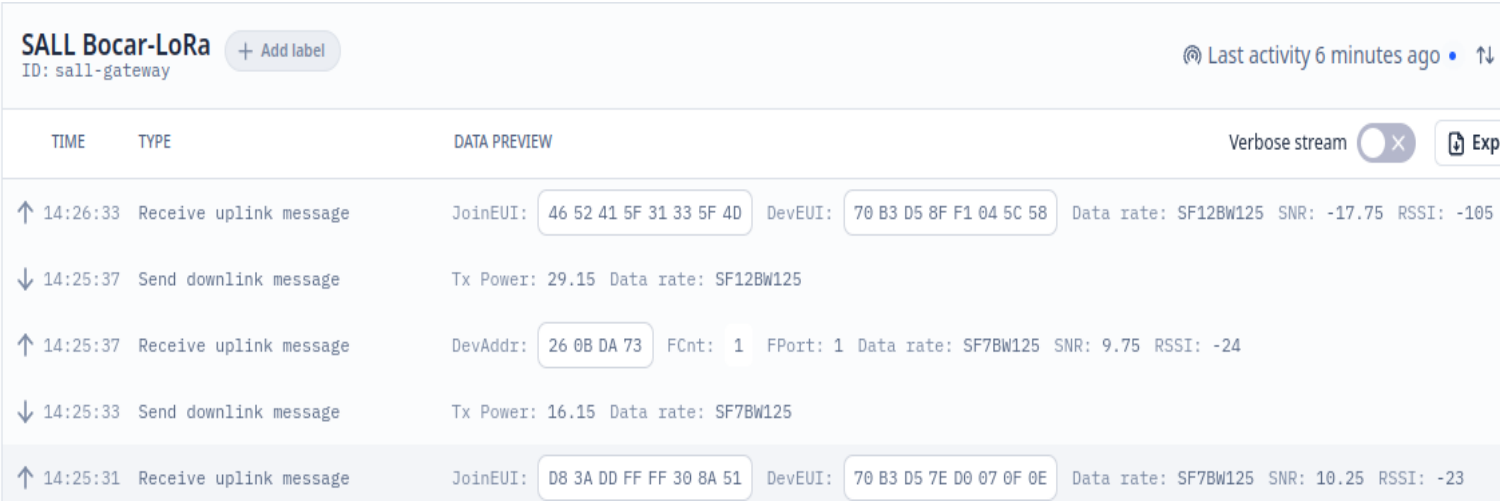


Figure56 : Join-Request et Join-Accept en OTAA

On peut tout voir sur la plateforme TTN où la passerelle reçoie les données et les renvoie aux serveurs.



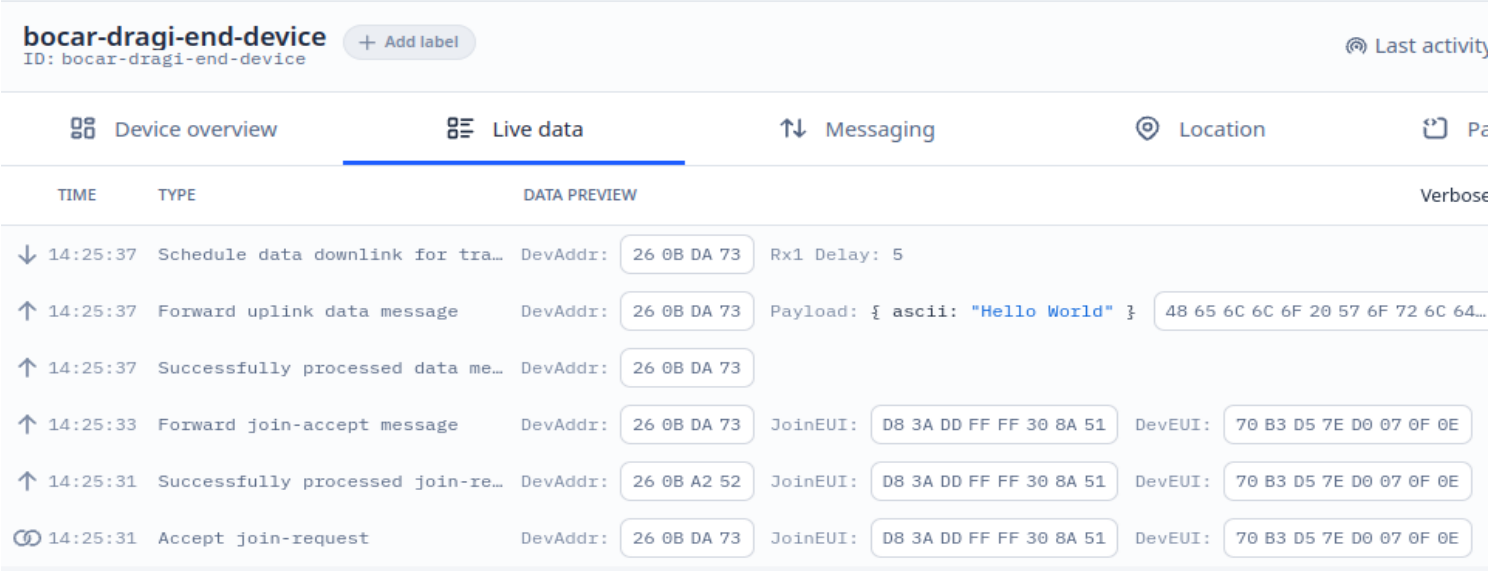


Figure57 : L'onglet live data de la passerelle et du dragino sur TTN

Apres l'envoie du dragino il sera necessaire de décoder le message à la réception pour une meilleure lecture.

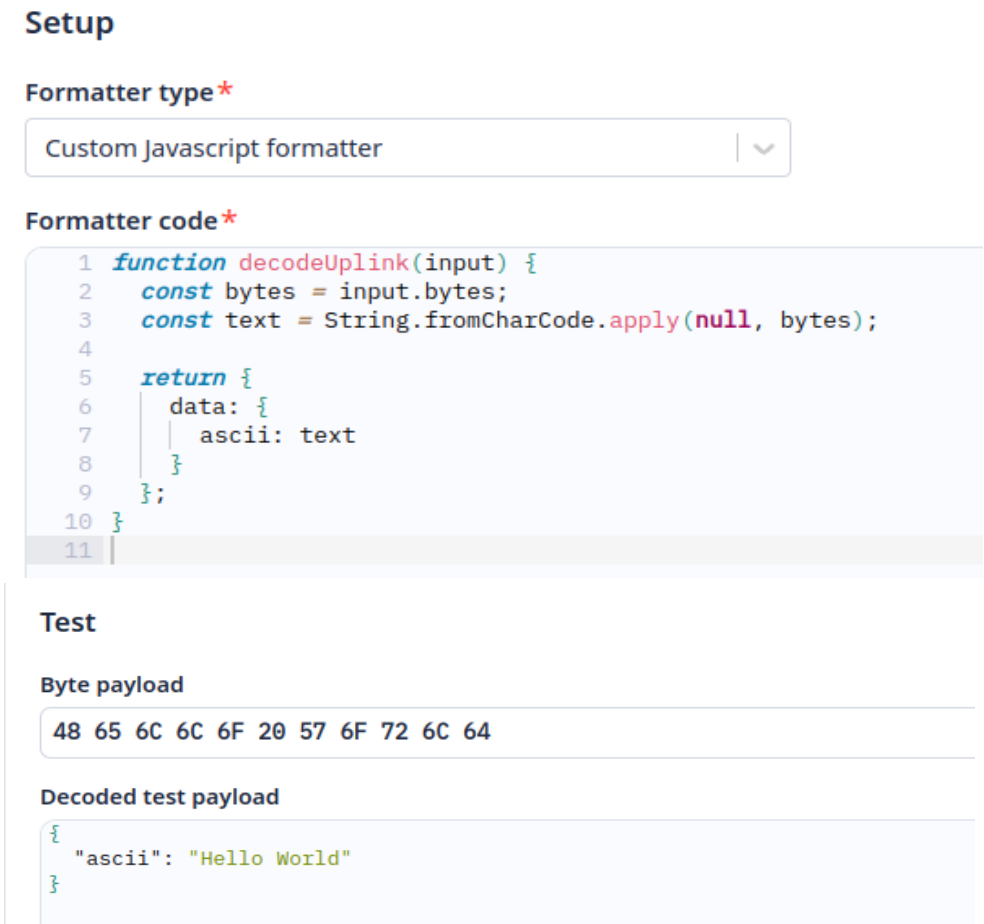


Figure58 : Décodage de l'hexadécimal en ascii

2.6 Essais

L'objectif de tout ce qu'on faisait jusque est de transmettre l'état de santé des plantes (saines ou malades) prédit par un model IA via une communication LoRa vers un réseau LoRaWAN (comme The Things Network - TTN).

2.6.1 Détection IA des plantes sur Raspberry Pi (Chapitre III)

Dans un premier temps, j'avais développé un script Python nommé prediction.py exécuté sur le Raspberry Pi. Ce script utilise un modèle de classification d'images (entrainé avec Teachable Machine) pour analyser des photos de feuilles de plantes capturées automatiquement.

2.6.2 Transmission LoRa avec module Dragino

Dans une deuxième phase, j'ai mis en place une transmission LoRa à l'aide d'un **module Dragino** connecté au Raspberry Pi. Pour cela, j'ai utilisé le projet single_chan_pkt_fwd, que j'ai adapté à ma configuration matérielle (broches SPI et GPIO).

J'ai compilé test.py, dont la fonction initiale était simplement d'envoyer un message de test « Hello World » vers une passerelle LoRa configurée pour communiquer avec **The Things Network (TTN)**. Cela permet de vérifier que les données envoyées par le Raspberry Pi sont bien reçues et visibles dans la console TTN.

2.6.3 Fusion des deux scripts

Après validation des deux étapes indépendantes, j'ai procédé à la fusion des scripts prediction.py et test.py avec une légère modification**.** L'objectif est que le résultat de la prédiction IA (sain ou malade) soit transmis automatiquement via LoRa au lieu d'envoyer « Hello World ». Je mets les deux scriptes fusionnés en annexe.

Ainsi, chaque fois qu'une photo est analysée, le résultat est envoyé sur TTN, où il est visible dans l'onglet "Live Data" du device connecté.

```
b2ap2@raspberrypi:~/dragino $ /bin/python /home/b2ap2/dragino/sendbylora.py
/home/b2ap2/dragino/dragino/SX127x/board_config.py:52: RuntimeWarning: This channel is already in use, continuing
 anyway.  Use GPIO.setwarnings(False) to disable warnings.
  GPIO.setup(BOARD.LED, GPIO.OUT)
Mode <- SLEEP
Mode <- FSK_STDBY
Mode <- SLEEP
Mode <- FSK_STDBY
Mode <- SLEEP
Recherche de nouvelles images à analyser...
12.jpg → malade (1.00)
Mode <- STDBY
Mode <- TX
Mode <- STDBY
Payload envoyé : malade:1.00 en 0:00:00.077721
Mode <- RXCONT
```

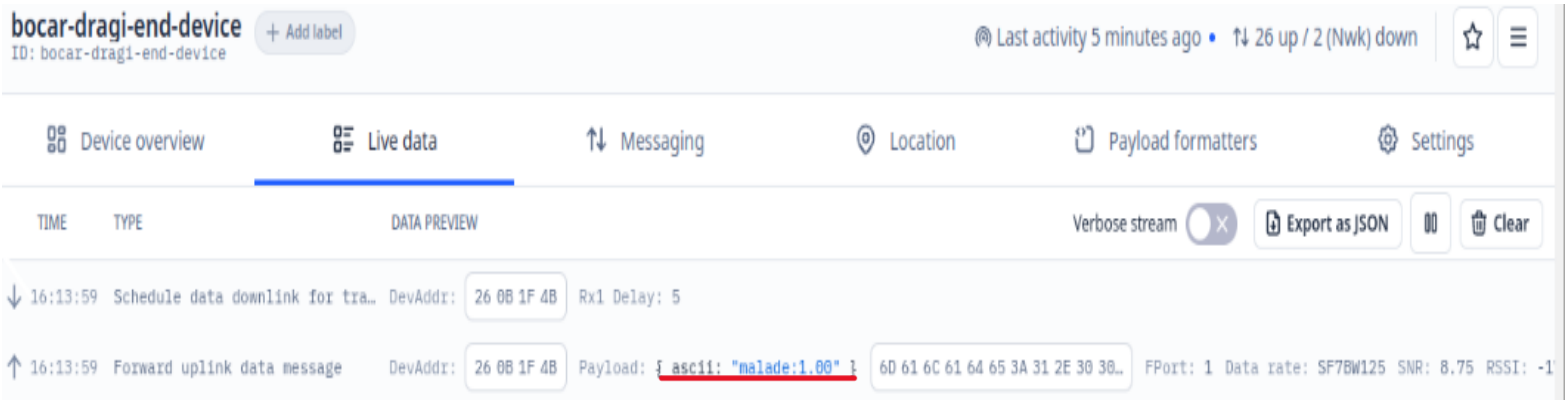


Figure59 : L'onglet Live data de la passerelle sur TTN

3. Migration de TTN vers ChirpStack et Intégration Node-RED

Dans le cadre du projet de surveillance agricole, j'utilisais initialement un module Dragino LoRa associé à une passerelle TTIG (The Things Indoor Gateway) connectée à The Things Network (TTN). Cette configuration permettait de recevoir des messages indiquant l'état de santé des plantes, sous forme de données telles que « sain » ou « malade ».

Ce pendant la passerelle TTIG conçue pour TTN ne permet pas une intégration facile avec ChirpStack qui est necessaire pour une intégration directe, simple et flexible avec Node-RED, un outil essentiel pour la visualisation locale.

3.1 Presentation de ChirpStack et Node-RED

-ChirpStack est un projet open-source disposant des deux services habituels : Network Server et Application Server, mais aussi d'un autre service appelé Gateway Bridge (figure 9).

Grâce au Gateway Bridge, ChirpStack est capable de communiquer avec plusieurs Packet Forwarder indépendamment du Network Server. Le Gateway Bridge convertit les différents formats des Packet Forwarder en un protocole MQTT commun utilisé par le Network Server de ChirpStack.

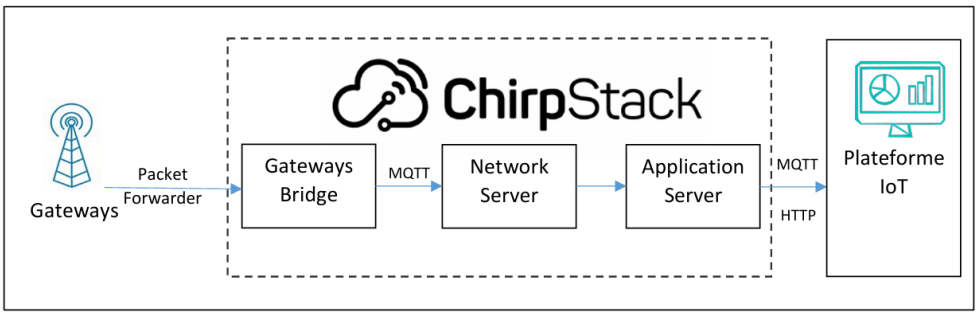


Figure60 : Architecture de ChirpStack

ChirpStack permet ainsi de faire la supervision des données reçues depuis une plateforme IoT par le biais des protocoles MQTT ou HTTP. ChirpStack dispose aussi d'une interface web permettant de faire la gestion des **passerelles** ainsi que des **end device**

-Node-RED est un outil de développement basé sur le web et conçu pour le prototypage rapide de systèmes connectés. Il permet de créer des flux de données visuels en glissant-déposant des "nœuds" qui représentent des actions, des capteurs, des traitements, ou des sorties.

Grâce à sa compatibilité avec **MQTT**, Node-RED est idéal pour interagir avec des serveurs comme ChirpStack. Dans mon projet, il est utilisé pour :

- **Récupérer les données MQTT** envoyées par ChirpStack,
- **Décoder et formater** les messages reçus,
- **Afficher dynamiquement les données** (état des plantes et heure de réception) dans un **tableau de bord web**.

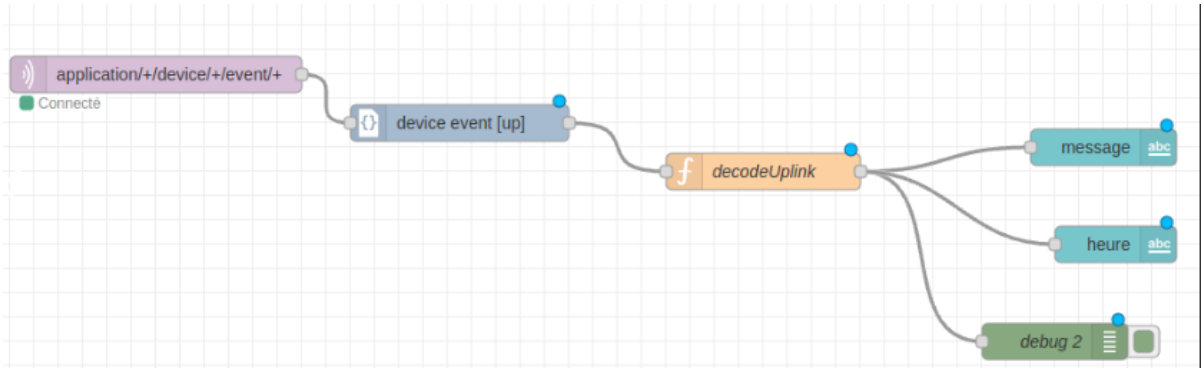


Figure61 : L'architecture du système sur Node-RED

3.2 Fonctionnement

Le système IoT que j'ai mis en place vise à surveiller l'état de santé des cultures à l'aide d'un capteur connecté LoRaWAN Dragino. Ce capteur transmet des données (comme « sain » ou « malade ») via une passerelle LoRaWAN connectée à ChirpStack, un Network Server local. Ensuite, ces données sont injectées dans Node-RED, qui les traite et les affiche sur un tableau de bord web.

Le processus suit ces étapes :

- **Le module Dragino** émet un message radio LoRa contenant des données codées en base64.
- **La passerelle LoRaWAN** reçoit le message et le transmet au serveur ChirpStack.
- ChirpStack décode le paquet LoRaWAN et le publie en MQTT sur le topic, Ce nœud capte tous les messages uplink envoyés par ChirpStack.

Figure62 : widget topic

Node-RED, abonné à ce topic MQTT, récupère le message, le **décode**, ajoute une **date** qui est utile pour tracer le moment exact de réception, puis l'envoie au tableau de bord via des widgets ui_text (Figure)

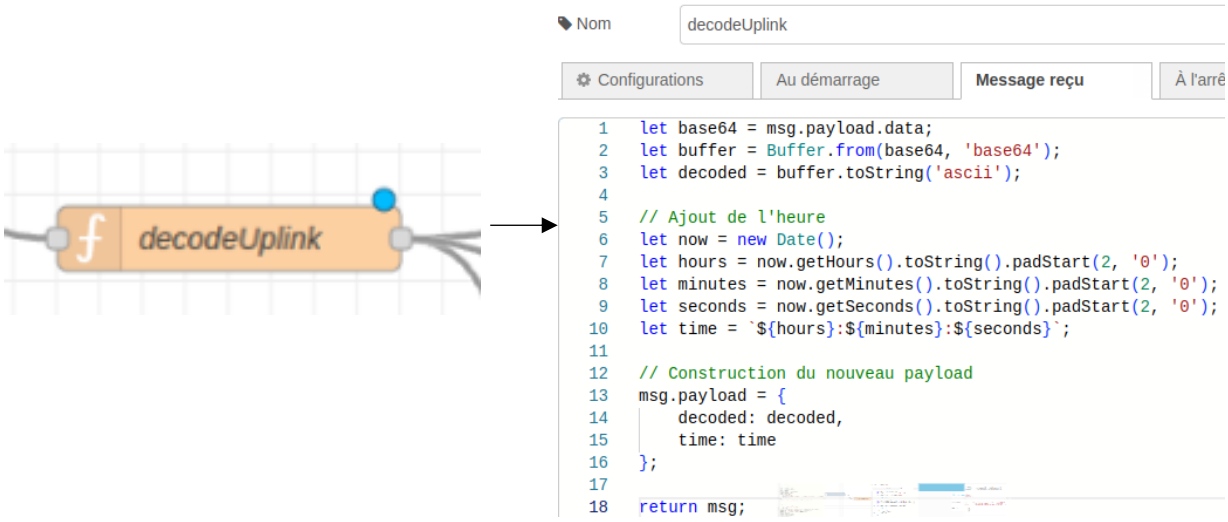


Figure63 :Decodage du payload en ascii et ajout de la date

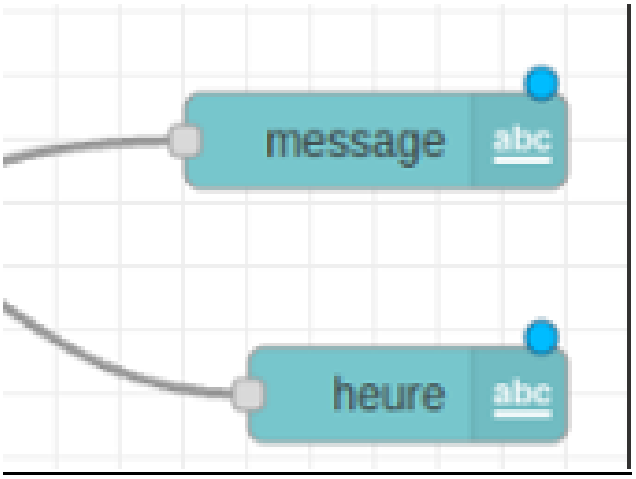


Figure64 : widget ui_text

Affichage sur le Dashboard

Deux nœuds ui_text affichent :

- Le message décodé (état de la plante),
- L'heure à laquelle le message a été reçu.

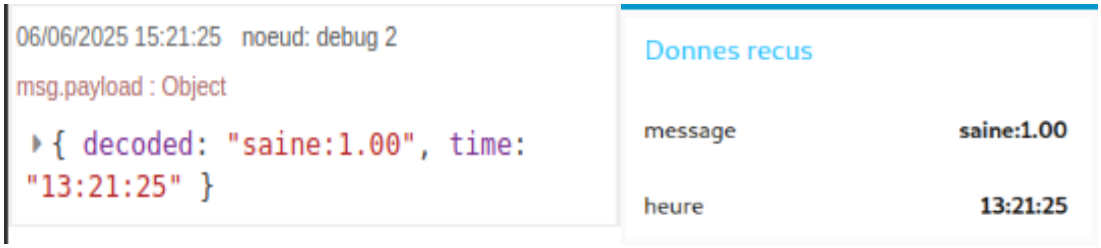


Figure65 : Affichage du message sur le tableau de bord

Ce tableau de bord est disponible via une URL du type : http://<IP_RaspberryPi>:1880/ui

Conclusion General

L'objectif principal de ce stage était d'automatiser et d'améliorer la surveillance des cultures à l'aide du robot potager FarmBot. Pour cela, j'ai d'abord assuré la mise en service complète du système en m'appuyant sur la documentation technique du fabricant. J'ai ensuite développé une solution basée sur l'intelligence artificielle pour détecter automatiquement certaines maladies végétales à partir d'images capturées par une caméra Raspberry Pi. Enfin, j'ai mis en place une transmission des données par LoRaWAN afin de permettre un suivi à distance de l'état de santé des plantes.

Les objectifs fixés au début du stage ont donc été atteints, tout en laissant entrevoir plusieurs pistes d'amélioration. Par exemple Il serait intéressant d'intégrer d'autres capteurs (humidité, température, etc.) afin d'enrichir le système de surveillance.

Ce stage m'a permis de développer des compétences en systèmes embarqués, en intelligence artificielle et en communication sans fil. J'ai apprécié le fait de contribuer à un projet aussi concret et utile, alliant technologies innovantes et automatisation de l'agriculture.

Reference

Source script Python Capture des photos : picamera2/examples/capture_image_full_res.py at main · raspberrypi/picamera2 · GitHub .

Documentation complète de la Raspberry : https://www.raspberrypi.com/documentation/

Mise en place de la caméra : https://youtu.be/GImeVqHQzsE?t=1

Récupération des paramètres du dragino : https://github.com/bbaranoff/ttn-gps

Documentation de la bibliothèque tflite_runtime : Référence API

Documentation de la bibliothèque Pillow **(PIL) :** Manuel

Documentation de la bibliothèque **numpy :** Docs

Script de l'implémentation du model IA : https://github.com/tensorflow/tensorflow/blob/master/tensorflow/lite/examples/python/label_image.py

Module caméra : PDF

Mon reseau TTN : https://eu1.cloud.thethings.network/console

Lora Shield Page produit : http://wiki.frontgino.com/index.php? Titre - Lora-Shield

LoRa Shield Hardware source : https://github.com/dragino/Lora

TTN Network Wiki : http://thethingsnetwork.org/wiki/Hardware/OverviewGateways-do-it-yourself

Forum de réseau TTN : http://forum.thethingsnetwork.org/

Sketch LoRa-Shield-TTN : https://github.com/dragino/Lora/blob/master/Lora%20Shield/Examples/lora-shield-ttn/lora-shield-ttn.ino

Plateforme du machine Learning : Teachable Machine

Annexes

Scripte en annexe.docx

Script du tableau de bord et de l’implémentation du model IA